

LIM2

16-BIT CORE

LIM2 Microprocessor Architecture

Official Technical Reference Manual and Core Specification

16-BIT

ALU & Microcode

Deterministic Execution

4 GB

32-Bit Address

Linear Flat Address Space

8-BIT

External Bus

Broad Hardware Interfacing

Technical Architecture Specification

Revision 2.1 — Year 2026

Status: **Active Architectural Hardware Standard**

Lead Hardware Architect:

Ivan Hniedash

github.com/ivan365

Contents

1 Introduction and Architectural Concept **1**
1.1 Basic Principles . . . 1

2 Data Path Organization **1**
2.1 Elimination of Internal Tri-State Buses . . . 1
2.2 Microcode Control . . . 2

3 Register Model and System Cell Architecture **2**
3.1 Architectural Register Summary . . . 3
3.2 Quantized 16-bit System Register Cells (REG_ID 0–255) . . . 3
3.3 Composite 32-bit Vector Register Aliases . . . 6

4 Flags and Control Register (FREG, 16-bit) **6**

5 Hardware Timer Subsystem (Timers 1–4 and TSR) **6**
5.1 Timer Setting Register (TSR, 16-bit, Cell 78) . . . 6
5.2 Functional Characteristics of Core Timers . . . 9
5.3 Instruction Cycle Latencies, Atomic Execution, and Carry-Over Semantics . . . 10
5.4 Timer Programming and Assembly Examples . . . 10

6 Hardware Reset (Reset) and Security Model **11**
6.1 Power-On Reset and RESET Signal Behavior . . . 11
6.2 Privilege Zones Operation (PZV) . . . 12

7 External Interface and Pinout **12**
7.1 Interface Signals . . . 12
7.2 Packaging Options . . . 12
7.3 Pinout Table . . . 12

8 Instruction Encoding Principles **14**

9 LIM2 Instruction Set Architecture **14**
9.1 Instruction Quick Catalog . . . 15
9.2 NOP — No operation and ALU residual latch reset. . . . 17
9.3 ADD — Addition with automatic carry cascading via ALU residual latch. . . . 18
9.4 SUB — Subtraction with automatic borrow cascading via ALU residual latch. . . . 19
9.5 MUL — Multiplication with upper word capture in ALU residual latch. . . . 20
9.6 DIV — Division with remainder capture in internal ALU latch. . . . 21
9.7 LMR — Load into register from memory or input; source config 0–3, size 8/16-bit. . . . 22
9.8 Assembler Convention: . . . 24
9.9 LRM — Store from register to memory or output; config 0–2, size 8/16-bit. . . . 25
9.10 Assembler Convention: . . . 27
9.11 LRR — Direct register-to-register copy via bypass MUX. . . . 28
9.12 CMP — Hardware operand comparison without modifying register values. . . . 29

9.13 JMP — Unconditional 32-bit jump: direct address (5 bytes), indirect, or via PTREG. 30
9.14 JCR — Conditional 32-bit jump on CMP result: $<$, $\leq$, $=$, $\neq$, $\geq$, $>$. 32
9.15 JO / JOB — Conditional 32-bit jump on overflow or borrow flag (OF/CF). 34
9.16 JZ / JNZ — Conditional 32-bit jump on zero flag (Zero Flag). 35
9.17 AND — Bitwise logical AND (conjunction). 36
9.18 OR — Bitwise logical OR (disjunction). 37
9.19 NAND — Bitwise negation of AND (Sheffer stroke). 38
9.20 NOR — Bitwise negation of OR (Peirce arrow). 39
9.21 NOT — Bitwise inversion (ones' complement). 40
9.22 XOR — Bitwise exclusive OR (addition modulo 2). 41
9.23 XNOR — Bitwise equivalence (inverted XOR). 42
9.24 INC — Hardware operand increment by 1. 43
9.25 DEC — Hardware operand decrement by 1. 44
9.26 PUSH — Push register onto stack space. 45
9.27 POP — Pop top of stack into target register. 46
9.28 BSL — Bitwise left shift by 1 with displaced bit capture. 47
9.29 BSR — Bitwise right shift by 1 with displaced bit capture. 49
9.30 MSP — Unified 16-bit quantized manipulation of system registers (FREG, SP, PTREG, IVT, PVZ, MEM). 50
9.31 MOD — Integer modulo computation (remainder of division). 54
9.32 INTR / INT — Software interrupt; invoke handler via vector IR0–IR15. 55
9.33 CALL — Subroutine call: push 32-bit PC to stack and branch to address. 56
9.34 RET — Return from subroutine with 32-bit PC restored from stack. 57
9.35 HALT — Hardware clock halt until interrupt or peripheral data ready. 58

10 Interrupt System and Exception Handling 59
10.1 Microarchitectural Interrupt Mechanism 59
10.2 Vector Table and Core Exceptions 60
10.3 Partitioning of Core Faults, Software Interrupts, and Hardware Timers 60

11 Memory Organization and Philosophy of Freedom 61
11.1 Fundamental Principle: Absolute Architectural Freedom 61
11.2 Direct Memory-Mapped I/O (MMIO) and Autonomous Controllers 61
11.3 Security Model: Dynamic Protection Without Hardcoded Constants 61

12 Practical System Assembly Examples 62
12.1 Architectural Conventions and Programming Model 62
12.2 Example 1: Complete System Cold-Boot Initialization (Startup / Bootloader) 63
12.3 Example 2: Hardware Divide-by-Zero Exception Handler 64
12.4 Example 3: System Interval Timer Handler (System Timer Tick) 64
12.5 Example 4: Software Operating System Trap (Syscall) 65
12.6 Example 5: High-Speed Memory Block Copy (Memcpy) 66

13 Architectural Roadmap and Reserved Modules (TODO) 66

1 Introduction and Architectural Concept

LIM2 is a 16-bit general-purpose microcoded microprocessor designed for strictly deterministic instruction execution and hardware portability. The specification does not mandate a fixed, recommended, or maximum clock frequency: it is determined entirely by the specific core implementation and the semiconductor process to which that implementation is targeted.

1.1 Basic Principles

Key Architectural Characteristics of LIM2

- **8-Bit External Data Bus ($D_0..D_7$):** Provides direct connection to a broad range of widely available Static RAM (SRAM), ROM (EEPROM/Flash), and standard peripherals. Word and pointer assembly is handled transparently by the interface controller.
- **32-Bit Linear Address Bus ($A_0..A_{31}$):** Direct physical address space of up to 4 GB without segmentation or page-table translation overhead.
- **Absolute Programmer Control:** The processor hardware never performs covert background context switching, never injects hidden constants, and enforces no implicit interrupts. The CPU executes exactly and only what is explicitly programmed.
- **Implementation-Defined Clocking:** Operational frequencies and timing parameters are selected upon core implementation and depend on circuit topology, silicon process, and external circuitry.
- **Peripheral Synchronization:** When wait-state monitoring is enabled, the processor holds the current micro-operation until the RF line asserts High. This naturally accommodates external device latencies without constraining the core clock.

2 Data Path Organization

2.1 Elimination of Internal Tri-State Buses

In conventional CPU designs, functional units share a common high-impedance (High-Z) tri-state bus. In the LIM2 architecture, internal tri-state buses are entirely eliminated:

1. All internal functional units are interconnected via dedicated directional digital multiplexers (MUX) and demultiplexers (DEMUX).
2. Data flow direction is governed strictly by the active control lines of the executing micro-instruction.
3. Operands are latched into accumulator buffer registers (ACCB) prior to entering the ALU, eliminating race conditions, glitches, and parasitic feedback loops.

The detailed circuit-level schematic organization of registers, MUX/DEMUX multiplexers, and the computational data path is shown in Figure 1.

▸ 2.2 Microcode Control

Each LIM2 assembly instruction is expanded by the control unit into a fixed sequence of chip-level micro-operations:

- Switching the register file multiplexer selectors;
- Configuring the ALU operation mode ($+$, $-$, $\times$, $/$, bitwise logic operations, shifts);
- Synchronizing memory read/write cycles with the external bus interface.

Instruction execution is strictly sequential: the next instruction is not fetched from memory until the microprogram of the current instruction has fully completed its execution cycle.

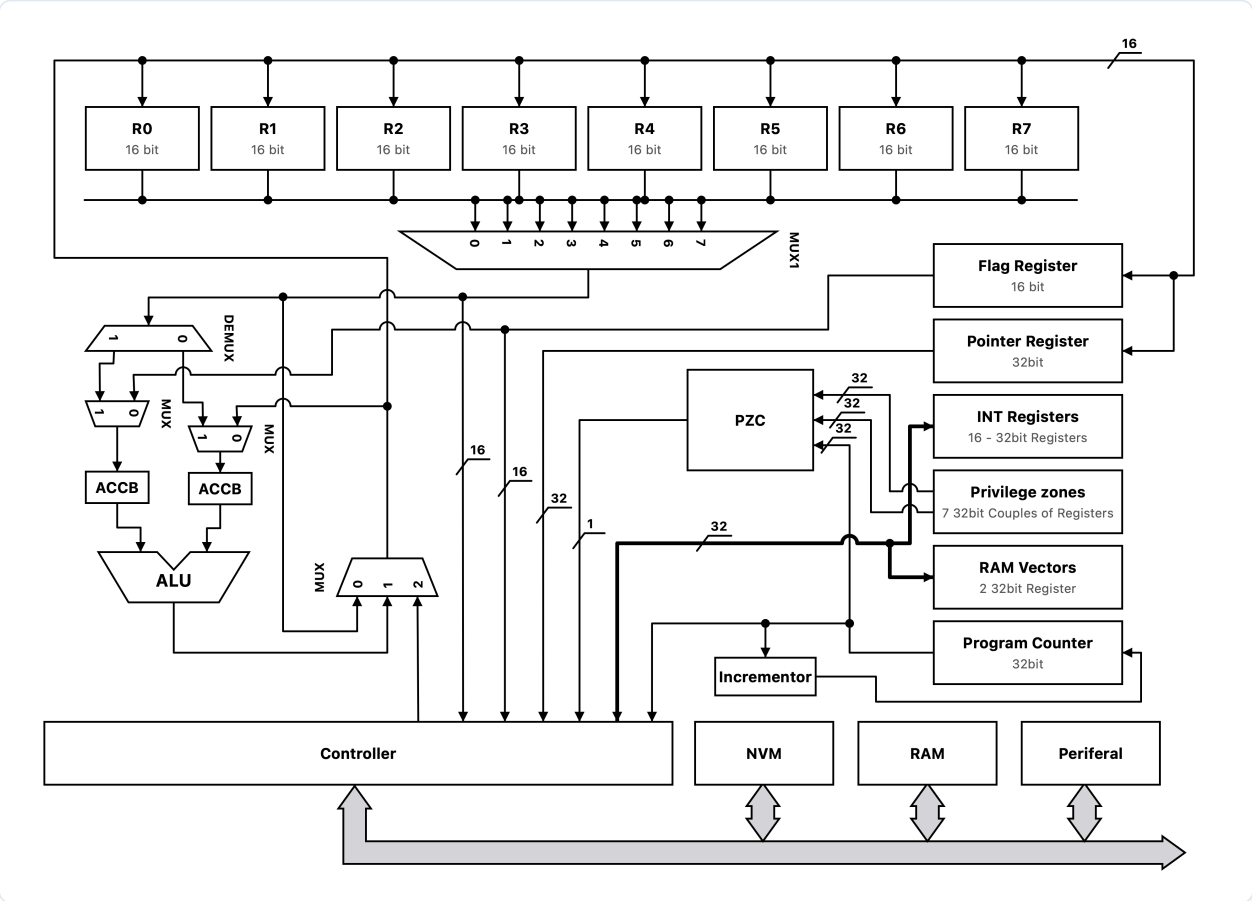


Figure 1: **LIM2 Microarchitectural Block Diagram:** Organization of register files R0–R7, directional MUX/DEMUX buses, ACCB latch buffers, ALU execution unit, Privilege Zone Controller (PZC), and system bus interface

3 Register Model and System Cell Architecture

The LIM M2 microarchitecture utilizes a strictly quantized, orthogonal register model partitioned into general-purpose registers, arithmetic accumulators, hardware execution pointers, and an indexed 8-bit space of 256 quantized 16-bit system cells (REG_ID 0–255).

3.1 Architectural Register Summary

Register	Bits	Reset	Type	Microarchitectural Role
R0 – R7	16	0x0000	General Purpose	Primary registers for ALU, logic, shift, and memory operands.
ACCA	16	0x0000	ALU Latch A	Internal ALU operand A buffer; holds intermediate computation state.
ACCB	16	0x0000	ALU Latch B	Internal ALU operand B buffer; feeds second operand to execution units.
PC	32	0x00000000	Instruction Ptr	Linear 32-bit physical Program Counter (up to 4 GB address space).
SP (SREG)	32	0x00000000	Stack Pointer	Hardware stack pointer for subroutine calls, interrupts, and push/pop.
PT (PTREG)	32	0x00000000	Base Pointer	Streaming memory pointer with automatic post-increment (AIP).
FREG	16	0x1800	Status & Control	Condition codes, ALU flags, interrupt enable (MIE), privilege (PRV).

3.2 Quantized 16-bit System Register Cells (REG_ID 0–255)

All privileged and configuration registers are addressed via the MSP instruction as quantized 16-bit cells. A full 8-bit address space (REG_ID $0–255$) is allocated. Registers wider than 16 bits are structured as consecutive pairs: the lower cell index represents the lower 16 bits (_lo), while the higher cell index represents the upper 16 bits (_hi). Both clean identifiers (freg, sp, ir0) and prefixed versions ($freg, $sp, $ir0) are recognized interchangeably by the assembler.

Table 3.1: Interrupt Vector Table (IVT) Cells (Cells 0–31, IRQ0–IRQ15)

Vector	Dec Cells	Hex Cells	Cell Aliases (Lo, Hi)	Functional Description and Handler Purpose
IRQ 0 (ir0)	0, 1	0x00, 0x01	ir0_lo, ir0_hi	Core Reset entry and Hardware Timer 0 ISR vector.
IRQ 1 (ir1)	2, 3	0x02, 0x03	ir1_lo, ir1_hi	Arithmetic Error Trap (division by zero / EM=1).
IRQ 2 (ir2)	4, 5	0x04, 0x05	ir2_lo, ir2_hi	System Call Trap (Syscall service entry via INT 2).
IRQ 3 (ir3)	6, 7	0x06, 0x07	ir3_lo, ir3_hi	External Peripheral Interrupt 3 (UART / Terminal console).
IRQ 4 (ir4)	8, 9	0x08, 0x09	ir4_lo, ir4_hi	Hardware Peripheral Interrupt 4 (General I/O controller).
IRQ 5 (ir5)	10, 11	0x0A, 0x0B	ir5_lo, ir5_hi	Hardware Peripheral Interrupt 5 (Storage / Disk controller).
IRQ 6 (ir6)	12, 13	0x0C, 0x0D	ir6_lo, ir6_hi	Hardware Peripheral Interrupt 6 (Network / Expansion bus).
IRQ 7 (ir7)	14, 15	0x0E, 0x0F	ir7_lo, ir7_hi	Hardware Peripheral Interrupt 7 (Real-time clock / periodic tick).
IRQ 8–11	16–23	0x10-0x17	ir8_lo..ir11_hi	Software and User Interrupt Vectors 8 through 11 (even: _lo, odd: _hi).
IRQ 12–15	24–31	0x18-0x1F	ir12_lo..ir15_hi	Dedicated Core Hardware Timers 1–4 (12: T1, 13: T2, 14: T3, 15: T4).

Note: Compact aliases iv0l..iv15h *are also accepted. Composite 32-bit identifiers* ir0..ir15 *expand into low and high cell pairs.*

Table 3.2: Privilege Protection Zones and Physical RAM Boundaries (Cells 32–63)

Zone / Vector	Dec Cells	Hex Cells	Boundary Aliases (Start / End)	Security Rules and Memory Isolation Window
Zone 0 (pz0)	32–35	0x20-0x23	pz0_start / pz0_end	Privilege Zone 0 window. In User Mode (PRV $= 0$), writes to cells 32–59 blocked outside this window.
Zone 1 (pz1)	36–39	0x24-0x27	pz1_start / pz1_end	Privilege Zone 1 window. Core kernel and supervisor code/data region.
Zone 2 (pz2)	40–43	0x28-0x2B	pz2_start / pz2_end	Privilege Zone 2 window. Privileged device drivers and memory manager.
Zone 3 (pz3)	44–47	0x2C-0x2F	pz3_start / pz3_end	Privilege Zone 3 window. Operating system shared services and IPC.
Zone 4 (pz4)	48–51	0x30-0x33	pz4_start / pz4_end	Privilege Zone 4 window. Task isolation boundary.
Zone 5 (pz5)	52–55	0x34-0x37	pz5_start / pz5_end	Privilege Zone 5 window. Task isolation boundary.
Zone 6 (pz6)	56–59	0x38-0x3B	pz6_start / pz6_end	Privilege Zone 6 window. Task isolation boundary.
RAM Base	60, 61	0x3C, 0x3D	ram_base	User RAM Physical Base (default: 0x00000000). Unrestricted across all modes.
RAM Limit	62, 63	0x3E, 0x3F	ram_limit	User RAM Physical Upper Limit (highest allowable physical RAM byte address).

Note: Individual 16-bit boundaries are accessible via _lo/_hi *(e.g.* pz0_start_lo*) or compact aliases* pz0sl, pz0sh / pz0el, pz0eh. *RAM bounds support* ram_bl, ram_bh / ram_ll, ram_lh.

Table 3.3: Control Registers, Hardware Timers, and System Extensions (Cells 64–255)

Register	Dec Cells	Hex Cells	Mnemonics and Aliases	Functional Description and Microcode Usage
T1_CNT	64, 65	0x40, 0x41	t1_cnt (t1_cnt_lo, hi)	32-bit Up-counter for Timer 1. Increments every clock cycle.
T1_MATCH	66, 67	0x42, 0x43	t1_match (t1_match_lo, hi)	32-bit Comparator match threshold. Triggers IRQ 12 and resets t1_cnt.
T1_TIME	68, 69	0x44, 0x45	t1_time (t1_time_lo, hi)	32-bit Elapsed time accumulator (increments upon each T1 match event).
T2_CNT	70, 71	0x46, 0x47	t2_cnt (t2_cnt_lo, hi)	32-bit Periodic Down-counter for Timer 2. Decrements every cycle.
T2_RELOAD	72, 73	0x48, 0x49	t2_reload (t2_reload_lo, hi)	32-bit Periodic reload period. Triggers IRQ 13 and reloads at zero.
T3_CNT	74, 75	0x4A, 0x4B	t3_cnt (t3_cnt_lo, hi)	32-bit One-shot Down-counter. Stops and triggers IRQ 14 upon zero.
T4_CNT	76	0x4C	t4_cnt	16-bit Instruction Quantum Down-counter (decrements per instruction).
T4_RELOAD	77	0x4D	t4_reload	16-bit Instruction quantum reload period. Triggers IRQ 15 at zero.
TSR	78	0x4E	tsr (timer_ctrl)	16-bit Timer Setting Register: enables (15:12), IRQ masks (11:8), status (7:4).
Reserved	79–127	0x4F-0x7F	—	49 Reserved cells for future core extensions and MMU tables.
FREG	128	0x80	freg (flags, sr)	16-bit Status and Control Flags: ZF, CF, OF, EM, CMP, CLK, SSRC, MIE, DWR, PRV, AIP, DBIT, SMOD, INTR.
SP (SREG)	129, 130	0x81, 0x82	sp (sp_lo, sp_hi, spl, sph)	32-bit Hardware Stack Pointer. Cell 129: SP[15:0] (_lo), Cell 130: SP[31:16] (_hi).
PT (PTREG)	131, 132	0x83, 0x84	ptreg (ptreg_lo, ptreg_hi, ptl, pth)	32-bit Streaming Base Pointer. Cell 131: PT[15:0] (_lo), Cell 132: PT[31:16] (_hi). Auto-incremented if AIP=1.
Reserved	133–255	0x85-0xFF	—	123 Reserved cells for hardware peripherals.

▸ 3.3 Composite 32-bit Vector Register Aliases

To maximize programmer convenience, the LIM M2 assembler transparently recognizes 32-bit register identifiers (with or without $) and expands single MSP.ld instructions into sequential 16-bit operations:

$$\texttt{MSP.ld sp, 0x0007FFFC} \implies \begin{cases} \texttt{MSP.ld sp_lo, 0xFFFC} & (\text{Cell } 129, \text{Low 16 bits}) \\ \texttt{MSP.ld sp_hi, 0x0007} & (\text{Cell } 130, \text{High 16 bits}) \end{cases}$$

Supported composite identifiers include:

- **sp / sreg:** 32-bit Hardware Stack Pointer (Cells 129 and 130).
- **ptreg / pt:** 32-bit Base Pointer (Cells 131 and 132).
- **ir0 – ir15:** 16 Interrupt Vectors IR0–IR15 (Cells $2k$ and $2k+1$).
- **pz0_start – pz6_end (pv0 – pv13):** 7 Privilege Zone boundary pairs.
- **ram_base / ram_limit:** Physical RAM boundaries (Cells 60/61 and 62/63).
- **t1_cnt, t1_match, t1_time:** 32-bit Timer 1 counters and comparator (Cells 64/65, 66/67, 68/69).
- **t2_cnt, t2_reload:** 32-bit Timer 2 counter and periodic reload threshold (Cells 70/71, 72/73).
- **t3_cnt:** 32-bit Timer 3 one-shot down-counter (Cells 74/75).

4 Flags and Control Register (FREG, 16-bit)

The FREG register combines arithmetic condition flags, clocking configuration bits, interrupt enables, and core privilege mode.



Table 4.1: Bit Definition Map of Flags and Control Register (FREG)

Bit	Flag	Name	Functional Description
0	**ZF**	Zero Flag	Set to 1 if the operation result equals zero.
1	**CF**	Carry Flag	Carry / borrow flag in arithmetic operations.
2	**OF**	Overflow Flag	Two's complement signed overflow indicator.
3	**EM**	Error Math	Hardware latch for divide-by-zero and arithmetic faults.
4–5	**CMP**	Compare Result	Comparator status: `00` = no comparison, `01` = $<$, `10` = $=$, `11` = $>$.
6–7	**CLK**	Clock Mode	Clock mode or prescaler configuration; frequencies depend on core implementation.
8	**SSRC**	Shift Source	Inflow bit source during linear shift (`SMOD=0`): 0 = zero, 1 = bit from `DBIT`.
9	**MIE**	Master Int Enable	External maskable interrupt enable. Software and internal faults remain active unconditionally.
10	**DWR**	Disable Wait Ready	When 0, CPU pauses microcode waiting for RF pin. When 1, wait state is disabled.
11	**PRV**	Privilege Mode	Execution mode: 0 = User Mode, 1 = Supervisor Mode.
12	**AIP**	Auto-Inc PTREG	Hardware auto-increment of `PT` on memory access (+1, +2, +4).
13	**DBIT**	Displaced Bit	Displaced bit latch from shifter (captured in hardware on all shifts).
14	**SMOD**	Shift Mode	Shift mode: 0 = linear (source selected by SSRC), 1 = circular ("bit garland": displaced bit recirculates to opposite end).
15	**INTR**	In Interrupt	Interrupt active flag (prevents nested interrupt corruption).

5 Hardware Timer Subsystem (Timers 1–4 and TSR)

The LIM2 architecture incorporates four hardware timers integrated directly into the core microarchitecture and memory-mapped to the 15 system extension cells 64–78 (0x40–0x4E). These timers provide deterministic high-resolution system profiling, real-time second/millisecond accumulation, periodic operating system scheduler ticks, single-shot countdown events, and instruction-quantum based multitasking preemption without cycle jitter.

All timers are centrally controlled and monitored through the 16-bit **Timer Setting Register** (TSR, Cell 78). Dedicated interrupt lines are assigned to vectors 12 through 15.

▸ 5.1 Timer Setting Register (TSR, 16-bit, Cell 78)

The TSR register manages hardware enable bits, independent interrupt masks, and status/pending latches for all four timers:

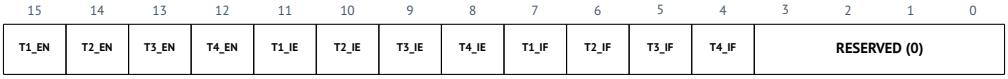


Table 5.1: Bit Definition Map of Timer Setting Register (TSR)

Bit	Flag	Name	Functional Description
15	**T1_EN**	Timer 1 Enable	Enables Timer 1 continuous increment on every core clock cycle.
14	**T2_EN**	Timer 2 Enable	Enables Timer 2 periodic down-counter.
13	**T3_EN**	Timer 3 Enable	Enables Timer 3 one-shot down-counter. Automatically cleared to 0 on timeout.
12	**T4_EN**	Timer 4 Enable	Enables Timer 4 instruction quantum counter (decrements per retired instruction).
11	**T1_IE**	Timer 1 IRQ Enable	Unmasks Timer 1 hardware interrupt request (IRQ 12).
10	**T2_IE**	Timer 2 IRQ Enable	Unmasks Timer 2 hardware interrupt request (IRQ 13).
9	**T3_IE**	Timer 3 IRQ Enable	Unmasks Timer 3 hardware interrupt request (IRQ 14).
8	**T4_IE**	Timer 4 IRQ Enable	Unmasks Timer 4 hardware interrupt request (IRQ 15).
7	**T1_IF**	Timer 1 IRQ Flag	Latched to 1 on comparator match (`t1_cnt == t1_match`). Cleared via MSP.offb.
6	**T2_IF**	Timer 2 IRQ Flag	Latched to 1 when down-counter hits zero (`t2_cnt == 0`). Cleared via MSP.offb.
5	**T3_IF**	Timer 3 IRQ Flag	Latched to 1 when one-shot counter reaches zero. Cleared via MSP.offb.
4	**T4_IF**	Timer 4 IRQ Flag	Latched to 1 when quantum counter reaches zero. Cleared via MSP.offb.
3–0	**RSV**	Reserved	Read as 0; writes ignored. Reserved for future timer cascades.

5.2 Functional Characteristics of Core Timers

Timer Subsystem Architectural Specifications

1. **Timer 1 (32-bit Up-counter & Real-Time Clock, IRQ 12):**
   - Consists of 32-bit counter `t1_cnt` (Cells 64, 65), 32-bit match threshold `t1_match` (Cells 66, 67), and 32-bit elapsed time register `t1_time` (Cells 68, 69).
   - While `T1_EN=1`, `t1_cnt` increments on each clock tick.
   - When `t1_cnt` $==$ `t1_match`, hardware resets `t1_cnt` $\leftarrow 0$, increments `t1_time` $\leftarrow$ `t1_time` $+ 1$, sets status flag `T1_IF=1`, and asserts **`IRQ 12`** (if `T1_IE` $= 1$ and `FREG.MIE` $= 1$).
   - Setting `t1_match` equal to the processor clock frequency (e.g., $50\,000\,000$ for 50 MHz) allows `t1_time` to accumulate true real-time seconds; loading freq$/1000$ accumulates milliseconds.
2. **Timer 2 (32-bit Periodic Down-counter, IRQ 13):**
   - Consists of 32-bit counter `t2_cnt` (Cells 70, 71) and 32-bit reload value `t2_reload` (Cells 72, 73).
   - While `T2_EN=1`, `t2_cnt` decrements by 1 each clock cycle.
   - When `t2_cnt` $== 0$, hardware automatically reloads `t2_cnt` $\leftarrow$ `t2_reload`, sets `T2_IF=1`, and asserts **`IRQ 13`**. Provides continuous, jitter-free periodic ticks for periodic event handling.
3. **Timer 3 (32-bit One-shot Down-counter, IRQ 14):**
   - Consists of 32-bit counter `t3_cnt` (Cells 74, 75). Requires no dedicated reload register.
   - The programmer writes the desired delay directly into `t3_cnt` and asserts `T3_EN=1`.
   - Decrements on every cycle while `t3_cnt` > 0.
   - Upon reaching 0, the hardware automatically halts counting by clearing `T3_EN` $\leftarrow 0$, sets `T3_IF=1`, and asserts **`IRQ 14`**. The timer remains stopped until software rearms it.
4. **Timer 4 (16-bit Instruction Quantum Down-counter, IRQ 15):**
   - Consists of 16-bit counter `t4_cnt` (Cell 76) and 16-bit reload value `t4_reload` (Cell 77).
   - Unlike cycle-based timers, `t4_cnt` decrements **strictly once per instruction retired** by the CPU core, regardless of instruction multi-cycle latencies.
   - When `t4_cnt` $== 0$, hardware reloads `t4_cnt` $\leftarrow$ `t4_reload`, sets `T4_IF=1`, and asserts **`IRQ 15`**.
   - Provides guaranteed, jitter-immune execution time-slicing for preemptive multitasking operating systems.

▸ 5.3 Instruction Cycle Latencies, Atomic Execution, and Carry-Over Semantics

In real hardware silicon, instructions require a variable number of clock generator cycles depending on instruction complexity, memory bus transfers, ALU latencies, and operand addressing modes:

- **Single-cycle fast operations (1–2 cycles):** NOP, HALT, register increments/decrements (INC, DEC), and bit shifts (BSL, BSR).
- **ALU register operations (2–3 cycles):** ADD, SUB, AND, OR, XOR, NAND, NOR, XNOR, LRR, NOT, CMP.
- **Memory loads and stores (LMR, LRM):** 2–3 cycles for immediate literals; 4–6 cycles for 32-bit absolute addresses ([ADDR]); 2–4 cycles for streaming pointer ([PTREG]); 3–5 cycles for register-pair addresses ([Rh:Rl]).
- **Multi-cycle arithmetic:** 4–6 cycles for multiplication (MUL); 6–10 cycles for hardware division and modulo (DIV, MOD).
- **Branches and control flow:** 3–4 cycles for jumps (JMP, JCR, JO, JZ); 5–7 cycles for subroutine calls (CALL) and interrupts (INT).
- **System register management (MSP):** 2–4 cycles.

Non-Preemptible Instruction Execution and Zero-Loss Carry-Over

1. **Atomic Instruction Retirement:** The processor core **never interrupts an instruction mid-execution**. An instruction is always executed to 100% completion (registers written, memory updated, PC advanced) before timer interrupts are evaluated.
2. **Instruction-Invariant Scheduling (Timer 4):** Timer 4 is an instruction-quantum counter: it decrements **strictly by 1** per completed instruction, completely independent of how many clock cycles that instruction consumed (Δ_{cycles}).
3. **Timer 1 Up-Counter Excess Carry-Over:** When Δ_{cycles} advances `t1_cnt` past `t1_match`, the surplus cycles are not discarded:

$$\text{matches} = \lfloor \texttt{t1_cnt} / \texttt{t1_match} \rfloor, \quad \texttt{t1_time} \leftarrow \texttt{t1_time} + \text{matches}$$
$$\texttt{t1_cnt} \leftarrow \texttt{t1_cnt} \pmod{\texttt{t1_match}}$$

   The residual $\texttt{t1_cnt} \pmod{\texttt{t1_match}}$ remains in the counter, preserving cycle accuracy.
4. **Timer 2 Periodic Down-Counter Deficit Carry-Over:** When $\Delta_{\text{cycles}} > \texttt{t2_cnt}$, the counter crosses zero, reloads from `t2_reload`, and absorbs the deficit:

$$\text{excess} = \Delta_{\text{cycles}} - \texttt{t2_cnt}, \quad \text{rem} = \text{excess} \pmod{\texttt{t2_reload}}$$

$$\texttt{t2_cnt} \leftarrow \begin{cases} \texttt{t2_reload} & \text{if rem} = 0 \\ \texttt{t2_reload} - \text{rem} & \text{if rem} > 0 \end{cases}$$

   Every clock cycle is accounted for with zero cumulative timing drift.

▸ 5.4 Timer Programming and Assembly Examples

Timers are initialized and controlled entirely through the 16-bit quantized MSP instruction subsystem:

Initializing Timer 1 (1 kHz Real-Time Clock) and Timer 4 (OS Quantum Preemption)

```
; --- 1. Set Up Timer 1 for 1 kHz Tick on a 50 MHz Core ---
MSP.ld  t1_match, 50000        ; 50,000 cycles = 1 ms at 50 MHz
MSP.ld  t1_time, 0             ; Reset elapsed millisecond counter
MSP.ld  t1_cnt, 0              ; Reset cycle counter

; --- 2. Set Up Timer 4 for 1000-Instruction Preemptive Time Slice ---
MSP.ld  t4_reload, 1000        ; Quantum length: 1000 instructions
MSP.ld  t4_cnt, 1000           ; Initialize quantum counter

; --- 3. Enable Timers and Unmask Interrupts in TSR ---
; Bit 15 (T1_EN) + Bit 11 (T1_IE) = 0x8800
; Bit 12 (T4_EN) + Bit 8  (T4_IE) = 0x1100
; Combined TSR word = 0x9900
MSP.ld  tsr, 0x9900            ; Arm Timer 1 and Timer 4 simultaneously
```

One-Shot Countdown with Timer 3

```
; Set a one-shot timeout of 2,000,000 cycles
MSP.ld   t3_cnt, 2000000       ; Write 32-bit delay count (Cells 74, 75)
MSP.onb  tsr, 0x2200           ; Set T3_EN (bit 13) and T3_IE (bit 9)
; Core continues execution... when 2,000,000 cycles elapse:
; Hardware automatically deasserts T3_EN, sets T3_IF, and vectors to IRQ 14.
```

6 Hardware Reset (Reset) and Security Model

6.1 Power-On Reset and RESET Signal Behavior

Guaranteed Initial State After Reset (RESET)

Upon power-on or assertion of the external RESET signal:

1. All general-purpose registers (R0–R7), program counter PC, stack pointer SP, address pointer PT, and all system vectors (INT_VECx, PZV_VECx, MEM_VECx) are strictly cleared to **0x00000000**.
2. The flags register FREG is initialized to **0x1800**:
   - PRV = 1 (Supervisor Mode active: system is ready for initial programmer setup);
   - DWR = 0 (Wait for Ready enabled: core is protected from slow memory timing faults and awaits RF);
   - AIP = 1 (Auto-increment of pointer PT enabled).

3. The hardware imposes no fixed memory maps. The system programmer is responsible for populating RAM boundaries (MEM_VECx), privilege zones (PZV_VECx), and interrupt handlers (INT_VECx) during early initialization.

6.2 Privilege Zones Operation (PZV)

Hardware Rules for Privilege Zones (PZV)

- The processor provides 7 programmable hardware windows: $[\text{PZV_VEC}_{2n} \ldots \text{PZV_VEC}_{2n+1}]$.
- **Entering Supervisor Mode:** Attempting to set PRV = 1 inspects the current PC. If PC resides within one of the 7 zones, promotion to supervisor mode is granted. Setting PRV = 1 outside these zones triggers an illegal operation fault.
- **Execution and Exiting Zones:** Advancing PC past the zone boundary while PRV = 1 **generates no fault and does not clear the flag**. The processor completely trusts the software: retaining supervisor mode outside the zone is considered legitimate behavior if intended by the programmer.
- Responsibility for clearing PRV $\rightarrow$ 0 before jumping into untrusted user code rests entirely with the system programmer.

7 External Interface and Pinout

7.1 Interface Signals

- **A0 – A31:** 32 address bus lines (direct linear access to 4 GB memory).
- **D0 – D7:** 8 bidirectional data bus lines (direct drive and interfacing).
- **RD, WD:** Read and Write strobes, active low.
- **RF (Ready Flag):** External device ready input. When DWR $= 0$, CPU suspends microcode until RF goes high.
- **IA (Interrupt Accepted):** Interrupt acknowledgment output pulse.
- **RDI (Raw Direct Interrupt):** Non-maskable direct hardware interrupt input.
- **CLK0, CLK1:** Two-phase external core clock inputs.
- **RESET:** Hardware master reset input.
- **VCC, GND:** Logic power (+5V) and system ground.

7.2 Packaging Options

7.3 Pinout Table

To avoid routing the high-speed clock trace through the middle of the address bus group, the A31 and CLK0 pins are swapped. Pin assignments 1–50 are identical for DIP-50 and QFP-64; pins 51–64 of the QFP package are not connected (NC).

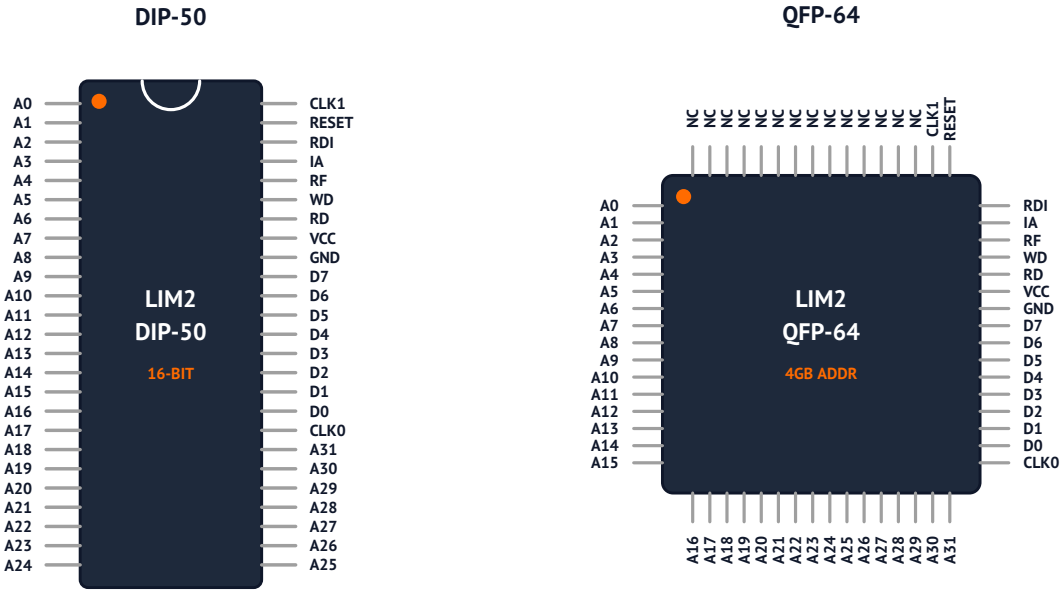


Figure 2: Top view of DIP-50 and QFP-64 packages. The orange dot marks Pin 1.

Pin(s)	Signal	Functional Description and Role
1–32	**A0-A31**	32-bit linear address bus.
33	**CLK0**	Primary phase of external clock synchronization.
34–41	**D0-D7**	8-bit bidirectional data bus.
42	**GND**	System power ground.
43	**VCC**	Primary core logic power (+5 V).
44	**RD**	Memory / peripheral read strobe (active low).
45	**WD**	Memory / peripheral write strobe (active low).
46	**RF**	Ready Flag: external device ready acknowledgment.
47	**IA**	Interrupt Accepted: interrupt receipt confirmation pulse.
48	**RDI**	Raw Direct Interrupt: non-maskable hardware interrupt.
49	**RESET**	System master hardware reset input.
50	**CLK1**	Secondary phase of external clock synchronization.
51–64	**NC**	Reserved unbonded pins (QFP-64 package only).

Table 1: Complete Pinout Table for DIP-50 and QFP-64 Packages

8 Instruction Encoding Principles

The LIM2 Instruction Set Architecture (ISA) is based on 32 primary opcodes with extended configuration bytes:

Machine Instruction Encoding Format

- **Instruction Length:** Variable — from 1 byte (simple control instructions NOP, HALT) up to 6 bytes (loading immediate 32-bit address/vector constants).
- **First Byte (Opcode):** Contains a 5-bit opcode field (yielding 32 primary instructions) and a 3-bit primary addressing / destination register field.
- **Configuration Byte (Config Byte):** For computational and transfer instructions, the second byte specifies microcode tuning (ALU mode, multiplexer routing, circular shift direction, flag condition inversion).
- **Immediate Operands:** Positioned immediately following the opcode or configuration byte: 1 byte for 8-bit values, 2 bytes for 16-bit words, 4 bytes for addresses and 32-bit vectors.

9 LIM2 Instruction Set Architecture

The instruction set defines 32 primary opcodes. The summary catalog is provided below, followed by dedicated reference specification pages for each instruction: each entry includes format diagrams, field breakdowns, formal microarchitectural semantics, flag side effects, and practical assembly examples. For instructions with multiple addressing or operand variants, dedicated sub-diagrams and tables are provided.

▸ 9.1 Instruction Quick Catalog

Opcode	Mnemonic	Class	Primary Function
00000	**NOP**	Control	No operation; clears ALU residual latch.
00001	**ADD**	Arithmetic	Addition with automatic carry cascading via ALU residual latch.
00010	**SUB**	Arithmetic	Subtraction with automatic borrow cascading via ALU residual latch.
00011	**MUL**	Arithmetic	Multiplication with upper word capture in ALU residual latch.
00100	**DIV**	Arithmetic	Integer division with remainder capture in internal ALU latch.
00101	**LMR**	Memory	Load into register from memory or input; config 0–3, size 8/16-bit.
00110	**LRM**	Memory	Store from register to memory or output; config 0–2, size 8/16-bit.
00111	**LRR**	Transfer	Direct register-to-register copy via bypass MUX.
01000	**CMP**	Comparison	Hardware operand comparison without modifying register values.
01001	**JMP**	Branch	Unconditional 32-bit jump: direct (5 bytes), indirect, or via PTREG.
01010	**JCR**	Branch	Conditional 32-bit jump on CMP result: $<$, $\leq$, $=$, $\neq$, $\geq$, $>$.
01011	**JO / JOB**	Branch	Conditional 32-bit jump on overflow or borrow flag (OF $= 1$ or CF $= 1$).
01100	**JZ / JNZ**	Branch	Conditional 32-bit jump on zero flag (bit 2: 0 = JZ, 1 = JNZ).
01101	**AND**	Logic	Bitwise logical AND (conjunction).
01110	**OR**	Logic	Bitwise logical OR (disjunction).
01111	**NAND**	Logic	Bitwise NAND (Sheffer stroke).

Opcode	Mnemonic	Class	Primary Function
10000	**NOR**	**Logic**	Bitwise NOR (Peirce arrow).
10001	**NOT**	**Logic**	Bitwise inversion (ones' complement).
10010	**XOR**	**Logic**	Bitwise exclusive OR (modulo-2 addition).
10011	**XNOR**	**Logic**	Bitwise equivalence (inverted XOR).
10100	**INC**	**Arithmetic**	Hardware operand increment by 1.
10101	**DEC**	**Arithmetic**	Hardware operand decrement by 1.
10110	**PUSH**	**Stack**	Push register onto hardware stack space.
10111	**POP**	**Stack**	Pop top of stack into target register.
11000	**BSL**	**Shifts**	Bitwise left shift by 1 with displaced bit capture.
11001	**BSR**	**Shifts**	Bitwise right shift by 1 with displaced bit capture.
11010	**MSP**	**System**	Special register manipulation (FREG, PTREG, SP, vectors).
11011	**MOD**	**Arithmetic**	Integer modulo computation (remainder of division).
11100	**INTR / INT**	**System**	Software interrupt; length 2 bytes, vector IR0–IR15.
11101	**CALL**	**Branch**	Subroutine call with 32-bit PC pushed to SP stack (modes 0–3).
11110	**RET**	**Branch**	Subroutine return with 32-bit PC popped from SP stack.
11111	**HALT**	**Control**	Hardware processor halt until interrupt or data ready.

▸ 9.2 NOP — No operation and ALU residual latch reset.

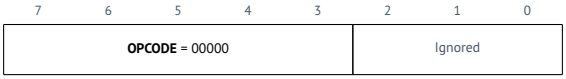
NOP

OPCODE: 00000

[Control]

Instruction Purpose:
No operation and ALU residual latch reset.

Instruction Format Diagram



Detailed Description and Microcode Semantics

The NOP (No Operation) instruction performs an idle microcode core cycle:

- **Data Path:** Initiates no read or write bus cycles on the system bus (RD=1, WD=1). Register file multiplexers remain neutral.
- **ALU:** Forcibly flushes internal accumulator buffer latches for remainder and carry cascading, preventing unintended residual effects from preceding mathematical operations.
- **Registers:** Values in general-purpose registers (R0–R7), stack pointer (SP), and address pointer (PT) remain unmodified. The program counter advances normally: PC $\leftarrow$ PC $+ 1$.

Length: 1 byte **Operands:** None **Mode:** User / Supervisor **Flags affected:** All FREG flags preserved (–)

Usage Example (LIM2 Assembly)

```
; 1. Guaranteed flush of ALU residual latch before cascaded calculation
NOP             ; Clear ALU residual register
ADD  $0, $0, $1 ; $0 = $0 + $1 (without previous carry interference)

; 2. Branch target alignment or timing delay (1 clock cycle)
NOP             ; Delay instruction fetch
```

Operational Notes: Invoking NOP before a chain of arithmetic operations guarantees a deterministic baseline state in the ALU. It is also used to generate fixed microsecond timing delays when polling slow external peripherals.

▸ 9.3 ADD — Addition with automatic carry cascading via ALU residual latch.

ADD

OPCODE: 00001

[Arithmetic]

Instruction Purpose:
Addition with automatic carry cascading via ALU residual latch.

Instruction Format Diagram



Detailed Description and Microcode Semantics

The ADD (Addition) instruction computes the sum of two register operands:

- **Data Path:** Computes the sum of numbers from the register pair designated by arguments `Operand 0` and `Operand 1`, targeting general-purpose registers **R0–R7**.
- **ALU Cascading:** If preceding arithmetic operations produced a carry stored in the ALU residual latch, it is automatically incorporated into the summation.
- **Registers:** The computed sum is stored in the destination register specified by `Result` (`R0-R7`).

Length: 2 bytes **Operands:** 3 **Mode:** User / Supervisor **Flags affected:** In case of arithmetic carry, bit CF is asserted to log. 1.

Usage Example (LIM2 Assembly)

```
ADD  $0, $1, $2      ; $0 = $1 + $2
```

Operational Notes: The ADD instruction adds values from two user registers. In the event of an arithmetic carry, the CF flag in FREG is asserted to logical 1.

▸ 9.4 SUB — Subtraction with automatic borrow cascading via ALU residual latch.

SUB

OPCODE: 00010

[Arithmetic]

Instruction Purpose:
Subtraction with automatic borrow cascading via ALU residual latch.

Instruction Format Diagram



Detailed Description and Microcode Semantics

The SUB (Subtraction) instruction computes the arithmetic difference between two registers:

- **Data Path:** Computes the difference between register operands designated by `Operand 0` and `Operand 1`, corresponding to general-purpose registers **R0–R7**.
- **ALU Borrow:** If preceding operations generated a borrow condition latched in the ALU residual buffer, it is automatically accounted for in the subtraction.
- **Registers:** The difference is stored in the destination register specified by `Result` (R0-R7).

Length: 2 bytes **Operands:** 3 **Mode:** User / Supervisor **Flags affected:** In case of borrow, bit CF is asserted to log. 1.

Usage Example (LIM2 Assembly)

```
SUB  $0, $1, $2      ; $0 = $1 - $2
```

Operational Notes: The SUB instruction subtracts the second user register from the first. When a borrow occurs, the CF flag in FREG is asserted to logical 1.

▸ 9.5 MUL — Multiplication with upper word capture in ALU residual latch.

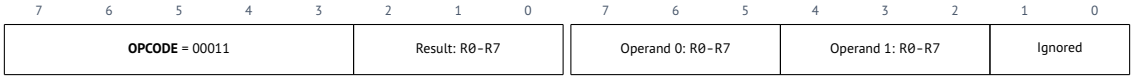
MUL

OPCODE: 00011

[Arithmetic]

Instruction Purpose:
Multiplication with upper word capture in ALU residual latch.

Instruction Format Diagram



Detailed Description and Microcode Semantics

The MUL (Multiply) instruction computes the product of two register operands:

- **Data Path:** Computes the product of operands from the register pair designated by `Operand 0` and `Operand 1`, pointing to user registers **R0–R7**.
- **Residual Accumulator:** The lower 16 bits of the 32-bit product are placed in the `Result` register, while the upper 16 bits are captured into the internal post-arithmetic ALU accumulator latch.
- **Registers:** The low word of the result is stored into `Result` (R0-R7).

Length: 2 bytes **Operands:** 3 **Mode:** User / Supervisor **Flags affected:** On arithmetic overflow, bit OF is asserted to log. 1.

Usage Example (LIM2 Assembly)

```
MUL  $0, $1, $2      ; $0 = $1 * $2
```

Operational Notes: The MUL instruction multiplies two values from user registers. If an overflow occurs, flags CF and OF are asserted to logical 1, and the post-arithmetic accumulator captures the upper 16 bits of the product.

The multiplication operation in the **LIM2** ALU is single-cycle in silicon, though the complete instruction takes multiple clock phases due to the microcoded core sequencing.

▸ 9.6 DIV — Division with remainder capture in internal ALU latch.

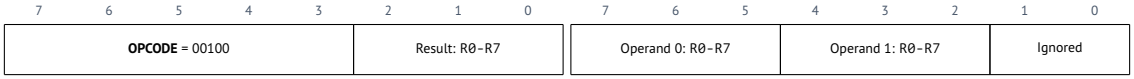
DIV

OPCODE: 00100

[Arithmetic]

Instruction Purpose:
Division with remainder capture in internal ALU latch.

Instruction Format Diagram



Detailed Description and Microcode Semantics

The DIV (Divide) instruction calculates the integer quotient of two register values:

- **Data Path:** Computes the integer quotient of the pair of registers designated by `Operand 0` and `Operand 1`, targeting user registers **R0–R7**.
- **Corner Cases:** The minimum result in integer division is 0, including cases where $1 > \frac{A}{B} > 0$. In such cases, ZF is asserted to logical 1 and the resulting quotient is 0. Division by zero triggers the EM math fault and vectors to ISR 1.

Length: 2 bytes **Operands:** 3 **Mode:** User / Supervisor **Flags affected:** On arithmetic overflow, bit OF is asserted to log. 1.

Usage Example (LIM2 Assembly)

```
DIV  $0, $1, $2      ; $0 = $1 // $2
```

Operational Notes: The DIV instruction performs integer division between user registers. If $1 > \frac{A}{B} > 0$, the result is set to 0 and the ZF flag is asserted to logical 1.

▸ 9.7 LMR — Load into register from memory or input; source config 0–3, size 8/16-bit.

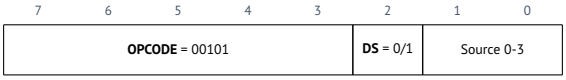
LMR

OPCODE: 00101

[Memory]

Instruction Purpose:
Load into register from memory or input; source config 0–3, size 8/16-bit.

Instruction Format Diagram

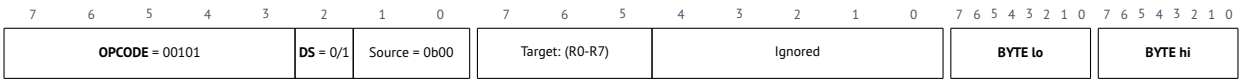


Detailed Description and Microcode Semantics

The LMR (Load from Memory to Register) instruction loads data from memory into a general-purpose register:

- **Data Path:** Loads a byte or word from memory into the selected destination register.
- **Modes and Configuration:** The DS (Data Size) bit specifies the operand width:
  $DS = 0 \rightarrow 1$ byte
  $DS = 1 \rightarrow 2$ bytes (word)
  This configuration does not modify the structure of subsequent arguments or addressing options.
- **Data Width Mnemonics:** To select operand width in LIM2 assembly, the `.b` / `.w` suffix is used.

Immediate Data Load Mode (Source = 0b00)



Immediate Data Load Mode

The LMR instruction when **SOURCE = 0b00**:

- **Data Path:** When **Source = 0b00**, the instruction loads an immediate byte/word directly into the target register: data follows immediately after the instruction opcode in memory, following Little-Endian order (lower byte at lower address).
- **Modes and Configuration:** The DS (Data Size) bit specifies the size:
  $DS = 0 \rightarrow 1$ byte
  $DS = 1 \rightarrow 2$ bytes (word)
- **Data Width Mnemonics:** Selected via `.b` or `.w` suffix in assembly.

Direct Address Load Mode (Source = 0b01)

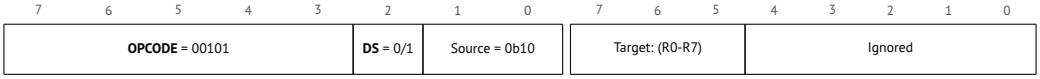


Direct Address Load Mode

The LMR instruction when **SOURCE = 0b01**:

- **Data Path:** When **Source = 0b01**, the instruction loads a byte/word from memory into the selected register using an immediate 32-bit physical address (4 bytes) following the instruction, adhering to Little-Endian order.
- **Modes and Configuration:** The DS (Data Size) bit specifies the operand size:
  $DS = 0 \rightarrow 1$ byte
  $DS = 1 \rightarrow 2$ bytes (word)
- This instruction variant occupies 6 bytes of memory (2 opcode/config bytes + 4 address bytes).
- **Data Width Mnemonics:** Selected via `.b` or `.w` suffix in assembly.

Pointer Register Indirect Load Mode (Source = 0b10)

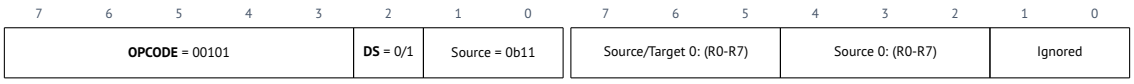


Pointer Register Indirect Load Mode

The LMR instruction when **SOURCE = 0b10**:

- **Data Path:** When **Source = 0b10**, the instruction loads a byte/word from memory into the destination register via the address contained in `Pointer Register (PTREG)`. This provides high efficiency for streaming and sequential block transfers.
- **Modes and Auto-Increment:** The DS bit defines data width:
  $DS = 0 \rightarrow 1$ byte
  $DS = 1 \rightarrow 2$ bytes (word)
  When auto-increment is enabled (`AIP=1` in `FREG`), accessing memory through `PTREG` automatically increments its value by the transfer size (+1 or +2), dramatically accelerating data stream processing.
- This instruction variant occupies 2 bytes of memory.
- **Data Width Mnemonics:** Selected via `.b` or `.w` suffix in assembly.

Register Pair Indirect Load Mode (Source = 0b11)



Register Pair Indirect Load Mode

The LMR instruction when **SOURCE = 0b11**:

- **Data Path:** When **Source = 0b11**, the instruction forms a 32-bit effective address from a pair of user registers according to: $(Rx \ll 16) \mid Ry$. The loaded data is stored into the register that held the upper address bits: $Rx = [(Rx \ll 16) \mid Ry]$.
- **Modes and Configuration:** The DS bit specifies the operand width:
  $DS = 0 \rightarrow 1$ byte
  $DS = 1 \rightarrow 2$ bytes (word)
  Because the register holding the upper address is overwritten by the loaded data, duplicated copies should be maintained if the pointer is reused.
- This instruction variant occupies 2 bytes of memory.
- **Data Width Mnemonics:** Selected via `.b` or `.w` suffix in assembly.

Length: 2–6 bytes **Operands:** 2–3 **Mode:** User / Supervisor **Flags affected:** On arithmetic overflow, bit OF is asserted to log. 1.

Usage Example (LIM2 Assembly)

```
;============================
; 1. Immediate data load into register
LMR.b $0, 0xFFFF          ; $0 = 0x00FF (byte mode selected)
;============================
; 2. Data load from direct 32-bit address
LMR.w $0, [0xF000]        ; $0 = Memory[0xF000 .. 0xF001]
;============================
; 3. Data load via pointer register PTREG
LMR $0, [$PTREG]       ; $0 = Memory[$PTREG] (auto-increments PTREG)
;============================
; 4. Data load via compound 32-bit register pair pointer
LMR $0, [$0:$1]     ; $0 = Memory[($0 << 16 | $1)] (word)
```

Operational Notes: The LMR instruction loads an 8-bit or 16-bit value from memory or an I/O device into a general-purpose register.

9.8 Assembler Convention:

Default Data Width Assumption

- When no explicit operand width suffix is specified in memory load/store instructions, the assembler defaults to word mode (word / 16-bit / 2 bytes), as this provides maximum performance and code clarity.

▸ 9.9 LRM — Store from register to memory or output; config 0–2, size 8/16-bit.

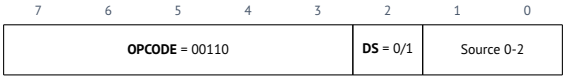
LRM

OPCODE: 00110

[Memory]

Instruction Purpose:
Store from register to memory or output; config 0–2, size 8/16-bit.

Instruction Format Diagram

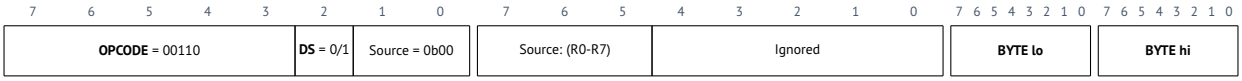


Detailed Description and Microcode Semantics

The LRM (Load to Memory from Register / Store Register to Memory) instruction stores data from registers into memory:

- **Data Path:** Stores a byte/word from a selected user register (or immediate constant) into system memory.
- **Modes and Configuration:** The DS (Data Size) bit specifies the operand width:
  $DS = 0 \rightarrow 1$ byte
  $DS = 1 \rightarrow 2$ bytes (word)
  This configuration does not modify the addressing parameters.
- **Data Width Mnemonics:** Selected via `.b` or `.w` suffix in assembly.

Immediate Data Store to Memory (Source = 0b00)



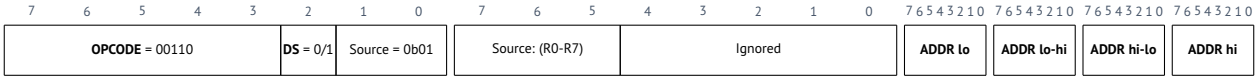
Immediate Data Store Mode

The LRM instruction when **SOURCE = 0b00**:

- **Data Path:** When **Source = 0b00**, the instruction writes immediate byte/word data into memory at the address currently pointed to by `Pointer Register (PTREG)`: the immediate value follows the instruction in memory (Little-Endian order).
- **Modes and Auto-Increment:** The DS bit defines data width:
  $DS = 0 \rightarrow 1$ byte
  $DS = 1 \rightarrow 2$ bytes (word)
  Writing data automatically advances `Pointer Register (PTREG)` by the written size (1 or 2 bytes).
- This instruction variant occupies 4 bytes of memory (for $DS = 1$).

- **Data Width Mnemonics:** Selected via `.b` or `.w` suffix in assembly.

Direct Address Store Mode (Source = 0b01)

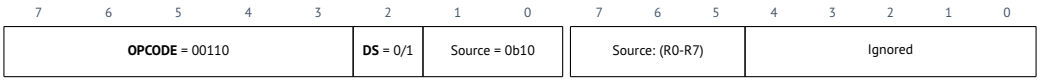


Direct Address Store Mode

The LRM instruction when **SOURCE = 0b01**:

- **Data Path:** When **Source = 0b01**, the instruction stores a byte/word from the selected user register into memory at the absolute 32-bit physical address (4 bytes) immediately following the instruction in Little-Endian format.
- **Modes and Configuration:** The DS bit defines data width:
  $DS = 0 \rightarrow 1$ byte
  $DS = 1 \rightarrow 2$ bytes (word)
- This instruction variant occupies 6 bytes of memory.
- **Data Width Mnemonics:** Selected via `.b` or `.w` suffix in assembly.

Pointer Register Indirect Store Mode (Source = 0b10)



Pointer Register Indirect Store Mode

The LRM instruction when **SOURCE = 0b10**:

- **Data Path:** When **Source = 0b10**, the instruction stores a byte/word from the chosen register into memory at the address specified by `Pointer Register (PTREG)`.
- **Modes and Auto-Increment:** The DS bit defines data width:
  $DS = 0 \rightarrow 1$ byte
  $DS = 1 \rightarrow 2$ bytes (word)
  When auto-increment is active, storing data automatically increases PTREG by the transferred width (+1 or +2 bytes), enabling fast sequential writes.
- This instruction variant occupies 2 bytes of memory.
- **Data Width Mnemonics:** Selected via `.b` or `.w` suffix in assembly.

Architectural Note: Absence of Compound Register-Pair Store Mode (0b11)

Unlike LMR, the LRM instruction **does not support** compound address mode via register pairs (**Source = 0b11**). The bit combination 11 is reserved. For indirect addressing, the target address is loaded into PTREG.

Length: 2–6 bytes **Operands:** 2 **Mode:** User / Supervisor **Flags affected:** Flags preserved (no effect on FREG).

Usage Example (LIM2 Assembly)

```
;============================
; 1. Store immediate data to memory via PTREG
LRM.b  [$PTREG], 0xFFFF  ; Memory[$PTREG] = 0x00FF (increments PTREG by 1)
;============================
; 2. Store register value to direct 32-bit address
LRM.w  [0xF000], $0      ; Memory[0xF000] = $0
;============================
; 3. Store register value via pointer PTREG
LRM  [$PTREG], $0     ; Memory[$PTREG] = $0 (increments PTREG by 2)
```

Operational Notes: The LRM instruction stores data from a user register or immediate constant into memory. Addressing via PTREG automatically advances the pointer by the written data width.

▸ 9.10 Assembler Convention:

Default Data Width Assumption

- When no explicit operand width suffix is specified in memory store instructions, the assembler defaults to word mode (word / 16-bit / 2 bytes).

▸ 9.11 LRR — Direct register-to-register copy via bypass MUX.

LRR

OPCODE: 00111

[Transfer]

Instruction Purpose:
Direct register-to-register copy via bypass MUX.

Instruction Format Diagram



Detailed Description and Microcode Semantics

The LRR (Load Register to Register) instruction copies data between registers:

- **Data Path:** Direct transfer of 16-bit word data from one user register to another via the internal bypass multiplexer. Register-to-register copying is always strictly 16-bit (word).

Length: 2 bytes **Operands:** 2 **Mode:** User / Supervisor **Flags affected:** Flags preserved (no effect on FREG).

Usage Example (LIM2 Assembly)

```
LRR $0, $1  ; $0 = $1 (Register Copy)
```

Operational Notes: The LRR instruction copies data from `Source ($0-$7)` into `Target ($0-$7)`.

▸ 9.12 CMP — Hardware operand comparison without modifying register values.

CMP

OPCODE: 01000

[Comparison]

Instruction Purpose:
Hardware operand comparison without modifying register values.

Instruction Format Diagram



Detailed Description and Microcode Semantics

The CMP (Compare) instruction performs an integer comparison between two register operands without altering their values:

- **Data Path:** Compares numbers in the designated registers and records the comparison outcome in bits 4–5 of FREG (CMP bits). Comparison is single-cycle in the ALU.
- **Result Encoding:** The CMP bit field states are defined as follows:
  $CMP = 0b00 \rightarrow$ No operation / invalid comparison
  $CMP = 0b01 \rightarrow A < B$
  $CMP = 0b10 \rightarrow A = B$
  $CMP = 0b11 \rightarrow A > B$

Length: 2 bytes **Operands:** 2 **Mode:** User / Supervisor **Flags affected:** Updates bits 4–5 (CMP) in FREG.

Usage Example (LIM2 Assembly)

```
; $0 = 10, $1 = 20 -> $0 < $1
CMP $0, $1        ; Flag Register CMP bits = 0b01
;=========================================================
; $0 = 10, $1 = 10 -> $0 = $1
CMP $0, $1        ; Flag Register CMP bits = 0b10
;=========================================================
; $0 = 20, $1 = 10 -> $0 > $1
CMP $0, $1        ; Flag Register CMP bits = 0b11
```

Operational Notes: The CMP instruction compares operands in user registers and records the result in the hardware CMP flag field of FREG for subsequent evaluation by conditional branch instructions.

▸ 9.13 JMP — Unconditional 32-bit jump: direct address (5 bytes), indirect, or via PTREG.

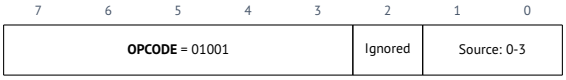
JMP

OPCODE: 01001

[Branch]

Instruction Purpose:
Unconditional 32-bit jump: direct address (5 bytes), indirect, or via PTREG.

Instruction Format Diagram



Detailed Description and Microcode Semantics

The JMP (Jump) instruction performs an unconditional control transfer:

- **Data Path:** Forcibly updates the program counter (PC) with the target 32-bit address.
- **Addressing Configurations:** The instruction supports multiple addressing modes:
  $Source = 0b00 \rightarrow$ Direct immediate 32-bit target address
  $Source = 0b01 \rightarrow$ Indirect address pointed to by immediate address in memory
  $Source = 0b10 \rightarrow$ Indirect address contained in Pointer Register (PTREG)
  $Source = 0b11 \rightarrow$ Indirect address formed by a user register pair

Direct Immediate Target (Source = 0b00)



Direct Immediate Jump Mode

The JMP instruction with **Source = 0b00**:

- **Data Path:** Unconditional branch to the 32-bit immediate physical address immediately following the instruction in Little-Endian format.
- **Instruction Length:** Occupies 5 bytes (1 byte opcode + 4 bytes target address).

Memory Indirect Target (Source = 0b01)



Memory Indirect Jump Mode

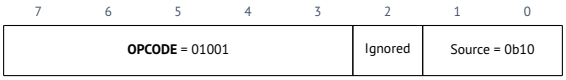
The JMP instruction with **Source = 0b01**:

- **Data Path:** Transfers execution to the 32-bit address stored in memory at the immediate

physical location specified after the instruction.

- **Instruction Length:** Occupies 5 bytes.

Pointer Register Indirect Target (Source = 0b10)



Pointer Register Indirect Jump Mode

The JMP instruction with **Source = 0b10**:

- **Data Path:** Transfers execution to the address currently loaded in `Pointer Register` (`PTREG`).
- **Instruction Length:** Occupies 1 byte.

Register Pair Indirect Target (Source = 0b11)



Register Pair Indirect Jump Mode

The JMP instruction with **Source = 0b11**:

- **Data Path:** Transfers execution to the composite 32-bit address formed by a user register pair: $(Source_h \ll 16) \mid Source_l$.
- **Instruction Length:** Occupies 2 bytes.

Usage Example (LIM2 Assembly)

```
JMP 0x00000000          ; Jump to address 0x00000000
;=============================================================================
JMP [0xFFFFFFFF]        ; Jump to address stored at memory location 0xFFFFFFFF
;=============================================================================
JMP [$PTREG]          ; Jump to address held in $PTREG
;=============================================================================
JMP [$0:$1]       ; Jump to address formed by registers ($0 << 16) | $1
```

Operational Notes: The JMP instruction unconditionally transfers control to the designated 32-bit address using direct, indirect, pointer-based, or register-pair addressing.

▸ 9.14 JCR — Conditional 32-bit jump on CMP result: $<, \leq, =, \neq, \geq, >$.

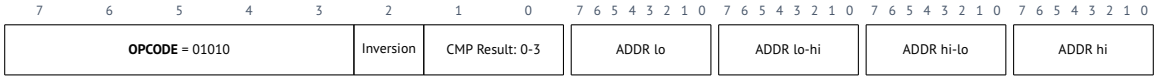
JCR

OPCODE: 01010

[Branch]

Instruction Purpose:
Conditional 32-bit jump on CMP result: $<, \leq, =, \neq, \geq, >$.

Instruction Format Diagram



Detailed Description and Microcode Semantics

The JCR (Jump on Condition Result) instruction conditionally branches based on the comparator state:

- **Data Path:** Transfers execution to the target address if the specified condition evaluates to true.
- **Operating Principle:** Branching occurs when the encoded CMP `Result` bits match the CMP flag field in FREG. The `Inversion` bit provides condition negation:
  $Inversion = 0 \mid CMP$ Result $== CMP$ bits in FREG $\longrightarrow$ jump
  $Inversion = 1 \mid CMP$ Result $== CMP$ bits in FREG $\longrightarrow$ fall-through
- **Condition Mappings:**
  $A < B \Longrightarrow Inversion = 0 \mid CMP_r = 01$
  $A \leqslant B \Longrightarrow Inversion = 1 \mid CMP_r = 11$
  $A = B \Longrightarrow Inversion = 0 \mid CMP_r = 10$
  $A \neq B \Longrightarrow Inversion = 1 \mid CMP_r = 10$
  $A > B \Longrightarrow Inversion = 0 \mid CMP_r = 11$
  $A \geqslant B \Longrightarrow Inversion = 1 \mid CMP_r = 01$
- The architecture uses immediate 32-bit target addresses for JCR (5 bytes total).

Usage Example (LIM2 Assembly)

```
JCR >=, 0x00000000      ; Jump to target if A >= B
;============================================================
JCR >,  0x00000000      ; Jump to target if A >  B
;============================================================
JCR <=, 0x00000000      ; Jump to target if A <= B
;============================================================
JCR <,  0x00000000      ; Jump to target if A <  B
;============================================================
JCR ==, 0x00000000      ; Jump to target if A == B
;============================================================
JCR !=, 0x00000000      ; Jump to target if A != B
```

Operational Notes: Performs a conditional branch based on the result of the preceding CMP instruction.

▸ 9.15 JO / JOB — Conditional 32-bit jump on overflow or borrow flag (OF/CF).

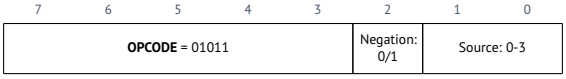
JO / JOB

OPCODE: 01011

[Branch]

Instruction Purpose:
Conditional 32-bit jump on overflow or borrow flag (OF/CF).

Instruction Format Diagram



Detailed Description and Microcode Semantics

Conditional jump instruction testing the overflow flag (OF, bit 2 of FREG) or carry/borrow flag (CF, bit 1 of FREG):

- **Bit 2 (Negation):** Overflow / borrow test polarity:
  - $\mathbf{0}$ (**JO / JOB**, *Jump on Overflow/Borrow*) — branch is taken on overflow or borrow (OF $= 1$ or CF $= 1$).
  - $\mathbf{1}$ (**JNO**, *Jump on No Overflow*) — branch is taken when no overflow occurs (OF $= 0$ and CF $= 0$).
- **Bits 1:0 (Source — Addressing):** $\mathbf{00}$ — direct 32-bit address (5 bytes); $\mathbf{01}$ — memory indirect [ADDR] (5 bytes); $\mathbf{10}$ — pointer indirect [PTREG] (1 byte); $\mathbf{11}$ — register pair indirect [Rh : Rl] (2 bytes).

Length: 1, 2, or 5 bytes **Operands:** 1 **Mode:** User / Supervisor **Flags affected:** Flags preserved (no effect on FREG).

Usage Example (LIM2 Assembly)

```
; 1. Check signed/unsigned arithmetic overflow after addition:
ADD  $0, $1, $2      ; $0 = $1 + $2
JO   MathOverflow            ; Jump to overflow handler (OF=1 or CF=1)

; 2. Check borrow after unsigned subtraction:
SUB  $0, $1, $2      ; If $2 > $1, borrow occurs (CF = 1)
JO   BorrowOccurred          ; Branch on borrow
JNO  ValidResult             ; Branch if operation completed normally
```

Operational Notes: The JO instruction detects arithmetic overflow or borrow conditions without requiring separate comparison instructions. The inverse mnemonic JNO branches when calculations completed without overflow.

▸ 9.16 JZ / JNZ — Conditional 32-bit jump on zero flag (Zero Flag).

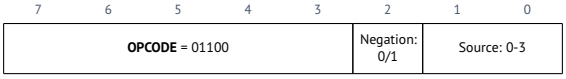
JZ / JNZ

OPCODE: 01100

[Branch]

Instruction Purpose:
Conditional 32-bit jump on zero flag (Zero Flag).

Instruction Format Diagram



Detailed Description and Microcode Semantics

Conditional jump instruction testing the zero flag ZF (bit 0 of FREG). Bit 2 serves as hardware condition negation:

- **Bit 2 (Negation):** Condition selection:
  - **0** (**JZ**, *Jump if Zero*) — branch taken if result is zero (ZF $= 1$).
  - **1** (**JNZ**, *Jump if Not Zero*) — branch taken on non-zero result (ZF $= 0$).
- **Bits 1:0 (Source — Addressing):** **00** — direct 32-bit address (5 bytes); **01** — memory indirect [ADDR] (5 bytes); **10** — pointer indirect [PTREG] (1 byte); **11** — register pair indirect [Rh : Rl] (2 bytes).

Length: 1, 2, or 5 bytes **Operands:** 1 **Mode:** User / Supervisor **Flags affected:** Flags preserved (no effect on FREG).

Usage Example (LIM2 Assembly)

```
; 1. Equality check via subtraction (JZ, Negation = 0):
SUB  $0, $1, $2      ; $0 = $1 - $2
JZ   ValuesEqual             ; Jump if $1 == $2 (ZF = 1)

; 2. Counted loop construct (JNZ, Negation = 1):
LoopHead:
    ; ... loop body ...
    DEC  $3              ; Decrement loop counter
    JNZ  LoopHead            ; Repeat loop while $3 != 0 (ZF = 0)
    HALT                     ; Exit loop upon reaching zero
```

Operational Notes: Instructions JZ and JNZ share the primary opcode 01100. The assembler encodes JZ with Negation $= 0$ (fires on ZF $= 1$), and JNZ with Negation $= 1$ (fires on ZF $= 0$).

▸ 9.17 AND — Bitwise logical AND (conjunction).

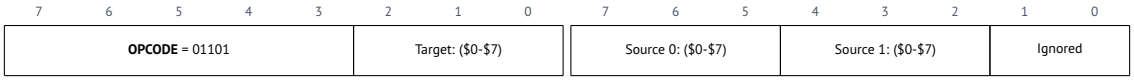
AND

OPCODE: 01101

[Logic]

Instruction Purpose:
Bitwise logical AND (conjunction).

Instruction Format Diagram



Detailed Description and Microcode Semantics

The AND instruction computes the bitwise logical conjunction of two 16-bit operands:

- **Data Path:** Operands are read from registers `Source 0` and `Source 1` and routed to the ALU logic unit. The result $Y = A \wedge B$ is written to register `Target`.
- **Flags Affected:** If the result is zero (`0x0000`), flag ZF is set to 1. Flag SF reflects the sign bit (bit 15). Flags CF and OF are cleared to 0.

Truth Table (AND):

Bit A (Src 0)	Bit B (Src 1)	Output Y (Target)	Logical Rule
0	0	0	Both bits are 0 → output 0
0	1	0	At least one bit is 0 → output 0
1	0	0	At least one bit is 0 → output 0
1	1	**1**	**Both bits are 1 → output 1 (active state)**

Length: 2 bytes **Operands:** 3 **Mode:** User / Supervisor **Flags affected:** ZF (1 if Y=0), SF (bit 15), CF=0, OF=0

Usage Example (LIM2 Assembly)

```
; Masking lower byte:
AND $0, $1, $2      ; $0 = $1 & $2 (mask)
; Zero test of specific bits:
AND $3, $3, $4      ; Bit coincidence test, sets ZF
```

Operational Notes: The AND instruction is used for applying bitwise masks (clearing selected bits while preserving others) and testing bit states in conjunction with JZ/JNZ.

▸ 9.18 OR — Bitwise logical OR (disjunction).

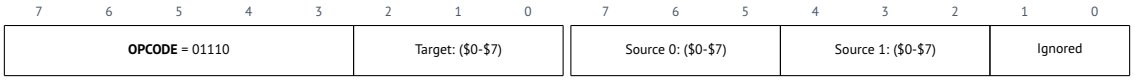
OR

OPCODE: 01110

[Logic]

Instruction Purpose:
Bitwise logical OR (disjunction).

Instruction Format Diagram



Detailed Description and Microcode Semantics

The OR instruction performs bitwise disjunction of two 16-bit operands:

- **Data Path:** Operands from `Source 0` and `Source 1` pass to the ALU. The bitwise logical sum $Y = A \vee B$ is stored in `Target`.
- **Flags Affected:** Flag ZF is 1 only if both inputs contain zeros (`0x0000`). Flag SF reflects bit 15; flags CF and OF are cleared.

Truth Table (OR):

Bit A (Src 0)	Bit B (Src 1)	Output Y (Target)	Logical Rule
0	0	0	Both bits are 0 → output 0 (quiescent state)
0	1	**1**	At least one bit is 1 → output 1
1	0	**1**	At least one bit is 1 → output 1
1	1	**1**	**Both bits are 1 → output 1**

Length: 2 bytes **Operands:** 3 **Mode:** User / Supervisor **Flags affected:** ZF (1 if Y=0), SF (bit 15), CF=0, OF=0

Usage Example (LIM2 Assembly)

```
; Setting bits by mask (forcing bits to logic 1):
OR $0, $0, $1       ; $0 = $0 | $1 (set mask bits from $1)
; Combining bit fields from two registers:
OR $2, $3, $4       ; $2 = $3 | $4
```

Operational Notes: The OR instruction sets individual bits to logic 1 without disturbing other bits, and merges bit fields from multiple sources.

▸ 9.19 NAND — Bitwise negation of AND (Sheffer stroke).

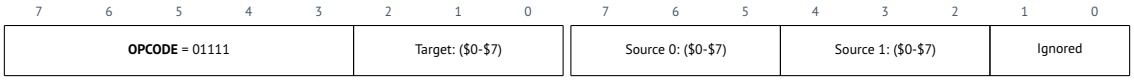
NAND

OPCODE: 01111

[Logic]

Instruction Purpose:
Bitwise negation of AND (Sheffer stroke).

Instruction Format Diagram



Detailed Description and Microcode Semantics

The NAND instruction executes a bitwise NAND operation (Sheffer stroke) on two 16-bit operands:

- **Data Path:** Operands from `Source 0` and `Source 1` enter the ALU logic block. The bitwise expression $Y = \overline{A \wedge B}$ is computed and stored in `Target`.
- **Flags Affected:** Flag ZF is 1 only if the result is zero (`0x0000`), which occurs when all bits of both operands are 1 (`0xFFFF`). Flag SF reflects bit 15; CF and OF are cleared.

Truth Table (NAND):

Bit A (Src 0)	Bit B (Src 1)	Output Y (Target)	Logical Rule
0	0	**1**	Both bits are 0 → output 1
0	1	**1**	At least one bit is 0 → output 1
1	0	**1**	At least one bit is 0 → output 1
1	1	0	**Both bits are 1 → output 0 (inverted conjunction)**

Length: 2 bytes **Operands:** 3 **Mode:** User / Supervisor **Flags affected:** ZF (1 if Y=0), SF (bit 15), CF=0, OF=0

Usage Example (LIM2 Assembly)

```
; Evaluating Sheffer stroke:
NAND $0, $1, $2        ; $0 = ~($1 & $2)
; Inverting an operand by supplying identical registers:
NAND $3, $1, $1        ; $3 = ~($1 & $1) = ~$1 (equivalent to NOT)
```

Operational Notes: The NAND instruction forms a functionally complete Boolean basis. In a single cycle, it computes inverted conjunction or unary negation when operands match.

▸ 9.20 NOR — Bitwise negation of OR (Peirce arrow).

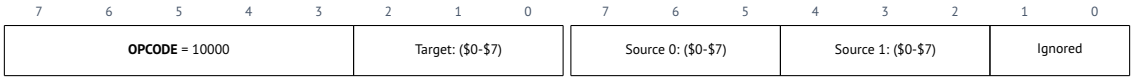
NOR

OPCODE: 10000

[Logic]

Instruction Purpose:
Bitwise negation of OR (Peirce arrow).

Instruction Format Diagram



Detailed Description and Microcode Semantics

The NOR instruction computes the bitwise NOR operation (Peirce arrow) between two 16-bit operands:

- **Data Path:** Operands in `Source 0` and `Source 1` are evaluated as $Y = \overline{A \vee B}$, and the result is latched into `Target`.
- **Flags Affected:** Flag ZF is 1 if the output is `0x0000`. Sign flag SF reflects bit 15; CF and OF are cleared.

Truth Table (NOR):

Bit A (Src 0)	Bit B (Src 1)	Output Y (Target)	Logical Rule
0	0	**1**	**Both bits are 0 → output 1 (active coincidence of zeros)**
0	1	0	At least one bit is 1 → output 0
1	0	0	At least one bit is 1 → output 0
1	1	0	Both bits are 1 → output 0

Length: 2 bytes **Operands:** 3 **Mode:** User / Supervisor **Flags affected:** ZF (1 if Y=0), SF (bit 15), CF=0, OF=0

Usage Example (LIM2 Assembly)

```
; Detecting concurrent absence of signals across bit fields:
NOR $0, $1, $2      ; $0 = ~($1 | $2) (1 only where both bits are 0)
; Bitwise inversion of an operand:
NOR $3, $1, $1      ; $3 = ~($1 | $1) = ~$1
```

Operational Notes: The NOR instruction yields logic 1 specifically in bit positions where both inputs are zero. Forms a universal Boolean basis.

▸ 9.21 NOT — Bitwise inversion (ones' complement).

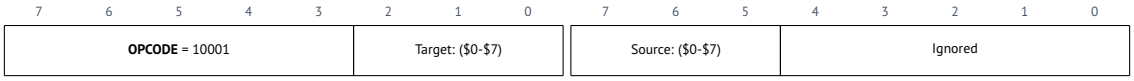
NOT

OPCODE: 10001

[Logic]

Instruction Purpose:
Bitwise inversion (ones' complement).

Instruction Format Diagram



Detailed Description and Microcode Semantics

The unary NOT instruction performs bitwise inversion (ones' complement) on the Source operand:

- **Data Path:** The 16-bit word from Source is passed to the ALU inverters. Every bit is inverted ($0 \to 1, 1 \to 0$), and the result $Y = \neg A$ is written to Target.
- **Flags Affected:** Flag ZF is 1 if the result is 0 (source was 0xFFFF). Flag SF reflects the inverted sign bit. Flags CF and OF are cleared.

Truth Table (NOT):

Input Bit A (Source)	Output Y (Target)	Logical Operation
0	**1**	Inversion of zero → bit is promoted to logic 1
1	**0**	Inversion of one → bit is cleared to logic 0

Length: 2 bytes **Operands:** 2 **Mode:** User / Supervisor **Flags affected:** ZF (1 if Y=0), SF (bit 15), CF=0, OF=0

Usage Example (LIM2 Assembly)

```
; Bitwise register inversion:
NOT $0, $1            ; $0 = ~$1
; Creating an inverted mask:
NOT $2, $2            ; Inverts all 16 bits of $2
```

Operational Notes: The NOT instruction inverts all 16 bits of the operand in a single execution cycle. Used for bitmask preparation, ones' complement calculation, and condition inversion.

▸ 9.22 XOR — Bitwise exclusive OR (addition modulo 2).

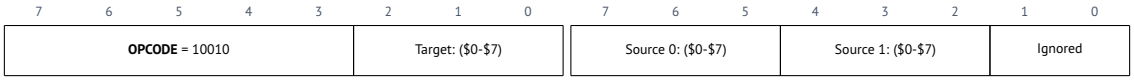
XOR

OPCODE: 10010

[Logic]

Instruction Purpose:
Bitwise exclusive OR (addition modulo 2).

Instruction Format Diagram



Detailed Description and Microcode Semantics

The XOR instruction performs bitwise modulo-2 addition over registers `Source 0` and `Source 1`:

- **Data Path:** Operands are routed to the modulo-2 adder. Output bit $Y = A \oplus B$ is 1 if input bits differ, and 0 if they match. The result is written to `Target`.
- **Flags Affected:** If the result is 0 (such as clearing a register via XOR `$0, $0, $0`), flag ZF is asserted to 1. Flag SF takes bit 15; CF and OF are cleared.

Truth Table (XOR):

Bit A (Src 0)	Bit B (Src 1)	Output Y (Target)	Logical Rule
0	0	0	Bits match → output 0
0	1	**1**	**Bits differ → output 1**
1	0	**1**	**Bits differ → output 1**
1	1	0	Bits match → output 0

Length: 2 bytes **Operands:** 3 **Mode:** User / Supervisor **Flags affected:** ZF (1 if Y=0), SF (bit 15), CF=0, OF=0

Usage Example (LIM2 Assembly)

```
; Fast register zeroing (idiomatic clear):
XOR $0, $0, $0          ; $0 = 0, ZF = 1
; Inverting specific bits via mask:
XOR $1, $1, $2          ; Inverts bits of $1 wherever $2 has 1s
```

Operational Notes: The XOR instruction provides the fastest method to clear a register in 1 cycle while asserting ZF=1. It is also fundamental in checksums and cryptographic routines.

▸ 9.23 XNOR — Bitwise equivalence (inverted XOR).

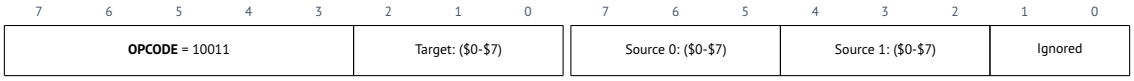
XNOR

OPCODE: 10011

[Logic]

Instruction Purpose:
Bitwise equivalence (inverted XOR).

Instruction Format Diagram



Detailed Description and Microcode Semantics

The XNOR instruction computes bitwise logical equivalence between `Source 0` and `Source 1`:

- **Data Path:** Operands enter the ALU equivalence unit. Output bit $Y = \overline{A \oplus B}$ is 1 when input bits match, and 0 when they differ. The result is written to `Target`.
- **Flags Affected:** Flag ZF is 1 if the result is 0 (all 16 bits were opposite). Flag SF takes bit 15; CF and OF are cleared.

Truth Table (XNOR):

Bit A (Src 0)	Bit B (Src 1)	Output Y (Target)	Logical Rule
0	0	**1**	**Both bits are 0 (match) → output 1**
0	1	0	Bits differ → output 0
1	0	0	Bits differ → output 0
1	1	**1**	**Both bits are 1 (match) → output 1**

Length: 2 bytes **Operands:** 3 **Mode:** User / Supervisor **Flags affected:** ZF (1 if Y=0), SF (bit 15), CF=0, OF=0

Usage Example (LIM2 Assembly)

```
; Bitwise equivalence check between two registers:
XNOR $0, $1, $2        ; $0 = ~($1 ^ $2) (1 in matching bit positions)
; Generating all-ones word (0xFFFF):
XNOR $3, $1, $1        ; $3 = 0xFFFF (identical operands always match)
```

Operational Notes: The XNOR instruction asserts 1 in bit positions where input operands match. When applied to identical arguments, it consistently evaluates to `0xFFFF`.

▸ 9.24 INC — Hardware operand increment by 1.

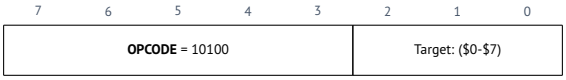
INC

OPCODE: 10100

[Arithmetic]

Instruction Purpose:
Hardware operand increment by 1.

Instruction Format Diagram



Detailed Description and Microcode Semantics

The INC instruction increments the 16-bit register value by 1 in hardware:

- **Data Path:** The `Target` register is fed to the internal ALU adder, where static constant 1 is added: Target $\leftarrow$ Target $+ 1$. The result is written back to the register in a single cycle.
- **Flags Affected:** Flag ZF is set to 1 on counter wrap-around (`0xFFFF` $\rightarrow$ `0x0000`). Flag SF reflects sign bit 15. Flag OF is asserted on signed overflow (`0x7FFF` $\rightarrow$ `0x8000`). Carry flag CF is preserved, maintaining multi-word addition chains.

Length: 1 byte **Operands:** 1 **Mode:** User / Supervisor **Flags affected:** ZF (1 on 0), SF (bit 15), OF (0x7FFF → 0x8000), CF unchanged

Usage Example (LIM2 Assembly)

```
; Increment loop counter:
INC $0                  ; $0 = $0 + 1
; Check counter wrap-around:
INC $1                  ; If $1 reached 0x0000, ZF = 1
JZ  0x00001000             ; Branch on counter wrap
```

Operational Notes: The single-byte encoding of INC minimizes instruction footprint during array traversals, pointer arithmetic, and loop execution.

▸ 9.25 DEC — Hardware operand decrement by 1.

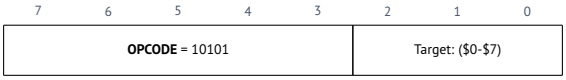
DEC

OPCODE: 10101

[Arithmetic]

Instruction Purpose:
Hardware operand decrement by 1.

Instruction Format Diagram



Detailed Description and Microcode Semantics

The DEC instruction decrements the 16-bit register value by 1 in hardware:

- **Data Path:** Subtracts 1 from the `Target` register: Target ← Target − 1. The result is written back in one machine cycle.
- **Flags Affected:** Flag ZF is asserted to 1 upon reaching zero (`0x0001` → `0x0000`). Flag SF reflects sign bit 15. Flag OF is asserted on signed underflow (`0x8000` → `0x7FFF`). Carry flag CF remains unchanged.

Length: 1 byte **Operands:** 1 **Mode:** User / Supervisor **Flags affected:** ZF (1 on 0), SF (bit 15), OF (0x8000 → 0x7FFF), CF unchanged

Usage Example (LIM2 Assembly)

```
; Down-counter loop:
DEC $0                  ; $0 = $0 - 1
JNZ 0x00000500             ; Repeat loop body while $0 != 0
```

Operational Notes: The single-byte DEC instruction is ideal for zero-terminated countdown loops in combination with JNZ, requiring no dedicated comparison (CMP).

▸ 9.26 PUSH — Push register onto stack space.

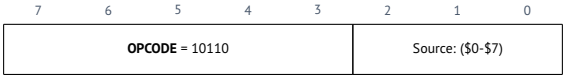
PUSH

OPCODE: 10110

[Stack]

Instruction Purpose:
Push register onto stack space.

Instruction Format Diagram



Detailed Description and Microcode Semantics

The PUSH instruction pushes a 16-bit word from register Source onto the hardware stack:

- **Data Path:** The 32-bit stack pointer SP is decremented by 2 bytes (SP $\leftarrow$ SP $-$ 2, stack grows downwards), and the 16-bit value from register Source ($0–$7) is written to memory: [SP] $\leftarrow$ Source.
- **Flags Affected:** The PUSH instruction does not alter FREG flags (ZF, SF, CF, OF remain unchanged).

Length: 1 byte **Operands:** 1 **Mode:** User / Supervisor **Flags affected:** Flags preserved (no effect on FREG)

Usage Example (LIM2 Assembly)

```
; Preserving registers before subroutine invocation:
PUSH $0                 ; SP -= 2, [SP] = $0
PUSH $1                 ; SP -= 2, [SP] = $1
CALL 0x00002000            ; Call subroutine with preserved register context
```

Operational Notes: The PUSH instruction preserves registers in stack frames. Used for argument passing, local frame allocation, and interrupt context protection.

▸ 9.27 POP — Pop top of stack into target register.

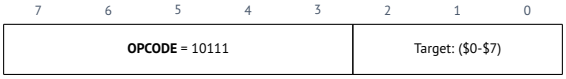
POP

OPCODE: 10111

[Stack]

Instruction Purpose:
Pop top of stack into target register.

Instruction Format Diagram



Detailed Description and Microcode Semantics

The POP instruction pops a 16-bit word from the top of the stack into the destination register `Target`:

- **Data Path:** A 16-bit word is read from memory at the physical address in SP into `Target` (Target $\leftarrow$ [SP]), after which SP is incremented by 2 bytes (SP $\leftarrow$ SP $+ 2$).
- **Flags Affected:** The POP instruction does not alter FREG flags.

Length: 1 byte **Operands:** 1 **Mode:** User / Supervisor **Flags affected:** Flags preserved (no effect on FREG)

Usage Example (LIM2 Assembly)

```
; Restoring registers in reverse order (LIFO):
POP  $1                 ; $1 = [SP], SP += 2
POP  $0                 ; $0 = [SP], SP += 2
RET                        ; Return from subroutine
```

Operational Notes: The POP instruction restores saved register values from the stack. Restoration order must be strictly inverse to the preservation order (PUSH).

▸ 9.28 BSL — Bitwise left shift by 1 with displaced bit capture.

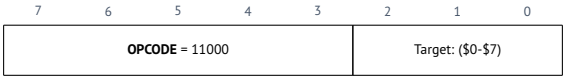
BSL

OPCODE: 11000

[Shifts]

Instruction Purpose:
Bitwise left shift by 1 with displaced bit capture.

Instruction Format Diagram



Detailed Description and Microcode Semantics

The BSL (Byte Shift Left) instruction shifts the 16-bit register `Target` left by strictly **1 bit** per machine cycle ($b_i \leftarrow b_{i-1}$):

- **Data Path and DBIT:** The most significant bit (b_{15}) is shifted out and always captured into the DBIT flag (bit 13 of FREG).
- **Inflow Bit Selection (b_0):** When SMOD $= 1$, circular rotation is active ("bit garland": $b_0 \leftarrow b_{15}$). When SMOD $= 0$ (linear shift), the inflow bit is governed by SSRC: zeros ($b_0 \leftarrow 0$) if SSRC $= 0$, or the value from DBIT ($b_0 \leftarrow$ DBIT) if SSRC $= 1$ (for multi-word 32/64-bit shifts or manual bit injection via MSP).
- **Multi-Bit Shifts:** To maintain low silicon complexity, the core omits a barrel shifter. Shifting by N bit positions is achieved by executing the instruction N times sequentially or within a loop.

Length: 1 byte **Operands:** 1 **Mode:** User / Supervisor **Flags affected:** DBIT (bit 13 of FREG is updated)

Usage Example (LIM2 Assembly)

```
; Logical shift left by 1 bit:
BSL  $0                 ; $0 = $0 << 1, bit b15 captured into DBIT
; Shift left by 2 bits (multiplication by 4):
BSL  $1
BSL  $1                 ; $1 = $1 << 2
; Dynamic multi-bit shift (shift 9 times):
Entry:
    LMR $0, 0x00001     ; $0 = 0b0000000000000001
    LMR $1, 0x9         ; Counter = 9
Loop:
    JZ  EndOfBSL
    DEC $1
    BSL $0
EndOfBSL:
    NOP                   ; Here $0 = 0b0000001000000000
```

Operational Notes: The BSL instruction shifts data by exactly 1 bit per cycle. When SMOD $= 1$, circular left rotation is performed; when SMOD $= 0$, SSRC $= 1$, cascaded 32-bit shifting is enabled via DBIT.

▸ 9.29 BSR — Bitwise right shift by 1 with displaced bit capture.

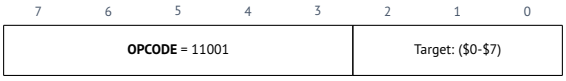
BSR

OPCODE: 11001

[Shifts]

Instruction Purpose:
Bitwise right shift by 1 with displaced bit capture.

Instruction Format Diagram



Detailed Description and Microcode Semantics

The BSR (Byte Shift Right) instruction shifts the 16-bit register `Target` right by strictly **1 bit** per machine cycle ($b_i \leftarrow b_{i+1}$):

- **Data Path and DBIT:** The least significant bit (b_0) is shifted out and always captured into the DBIT flag (bit 13 of FREG).
- **Inflow Bit Selection (b_{15}):** When SMOD $= 1$, circular right rotation is active ("bit garland": $b_{15} \leftarrow b_0$). When SMOD $= 0$ (linear shift), the inflow bit is selected by SSRC: zeros ($b_{15} \leftarrow 0$) if SSRC $= 0$, or the value from DBIT ($b_{15} \leftarrow$ DBIT) if SSRC $= 1$ (for multi-word cascading).
- **Multi-Bit Shifts:** Omitting a barrel shifter maximizes silicon simplicity. For shifting right by N positions, execute BSR sequentially N times or within a countdown loop.

Length: 1 byte **Operands:** 1 **Mode:** User / Supervisor **Flags affected:** DBIT (bit 13 of FREG is updated)

Usage Example (LIM2 Assembly)

```
; Logical shift right by 1 bit (least significant bit b0 captured in DBIT):
BSR  $0                 ; $0 = $0 >> 1
; Shift right by 2 bits (fast integer division by 4):
BSR  $2
BSR  $2                 ; $2 = $2 >> 2
; Shift right by 4 bits in a countdown loop:
Loop:
    JZ   EndBSR             ; Exit when count reaches 0
    DEC  $1             ; Decrement loop counter
    BSR  $0             ; Shift target register right by 1
EndBSR:
    NOP                     ; Resulting value in $0
```

Operational Notes: The BSR instruction shifts data right by 1 bit per cycle. With SMOD $= 1$, right circular rotation occurs; with SMOD $= 0$, SSRC $= 1$, multi-word cascading arithmetic is supported.

▸ 9.30 MSP — Unified 16-bit quantized manipulation of system registers (FREG, SP, PTREG, IVT, PVZ, MEM).

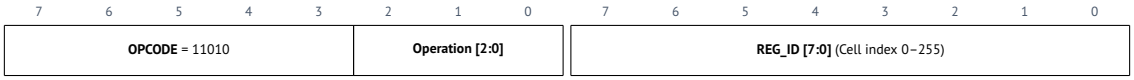
MSP

OPCODE: 11010

[System]

Instruction Purpose:
Unified 16-bit quantized manipulation of system registers (FREG, SP, PTREG, IVT, PVZ, MEM).

Instruction Format Diagram

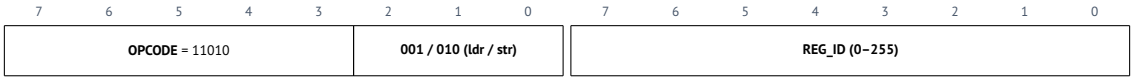


Detailed Description and Microcode Semantics

The MSP (Modify System Parameter) instruction (opcode 11010) provides a unified, strictly quantized 16-bit interface for reading, loading, and modifying all internal architectural and privileged registers:

- **Quantized 16-bit Cell Space:** Every system register is accessed as one or more 16-bit quantized cells indexed by an 8-bit REG_ID (0 through 255).
- **Sub-Operation Field (Bits [2:0] of Byte 0):** Encodes 8 specialized hardware sub-operations: direct 16-bit loading (ld), fast general register exchange via R7 (ldr7, str7), bitwise masks (and, or, xor), and explicit flag toggling (onb, offb).
- **Composite Multi-Word Registers:** All registers wider than 16 bits (such as 32-bit stack pointer SP, streaming pointer PTREG, interrupt vectors IR0-IR15, and privilege zones PV0-PV13) are structured as consecutive pairs: the lower cell index represents the lower 16 bits (_lo), and the higher cell index represents the upper 16 bits (_hi).
- **Linear Branching Compatibility:** Flow control instructions (CALL, RET, INT, JMP) continue to operate natively with full 32-bit linear address values without modification.

Register R7 Transfer Modes (ldr = 0b001, str = 0b010)



Register R7 Transfer Modes (2 Bytes)

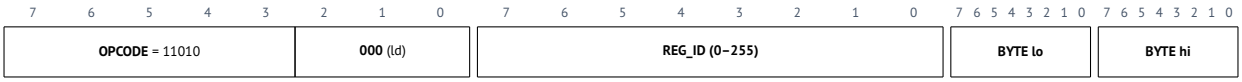
The MSP instruction in **R7 Exchange Modes** provides compact 2-byte register transfers between user register R7 and any internal system cell:

- **MSP.ldr REG_ID (Operation = 0b001, Byte 0 = 0xD1):** Transfers the 16-bit value from general-purpose register R7 into the designated system cell: SYS_REG[id] ← R7.
- **MSP.str REG_ID (Operation = 0b010, Byte 0 = 0xD2):** Reads the 16-bit value from the designated system cell directly into general-purpose register R7: R7 ← SYS_REG[id].
- **Microarchitectural Role:** General register R7 acts as the dedicated CPU hardware bridge to

save/restore flags (`freg`), stack pointers, and vectors to/from memory and stack (PUSH R7, POP R7) during fast context switching.

- **Instruction Length:** Occupies exactly **2 bytes** of program memory.

Immediate 16-bit Constant Load Mode (ld = 0b000)

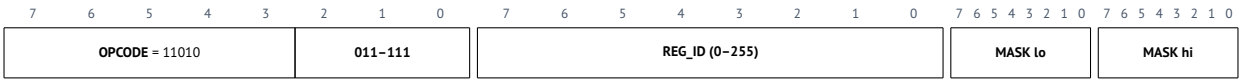


Immediate 16-bit Constant Load Mode (4 Bytes)

The MSP instruction when **Operation = 0b000** (`.ld`) loads an immediate 16-bit constant directly into a system cell:

- **Data Path:** Writes the 16-bit immediate word directly into the designated cell: SYS_REG[id] $\leftarrow$ imm16.
- **Byte Ordering:** Follows standard Little-Endian memory layout (`BYTE lo` at lower address, followed by `BYTE hi`).
- **Composite 32-bit Vector Loading:** When programming 32-bit registers, two sequential `MSP.ld` instructions are executed (first the lower 16 bits to cell $2k$, followed by the upper 16 bits to cell $2k+1$). The assembler provides macro-expansion aliases (e.g. `MSP.ld sp, 0x0007FFFC` expands into `sp_lo` and `sp_hi`).
- **Instruction Length:** Occupies exactly **4 bytes** of program memory.

Bitwise Logic and Mask Modification Modes (and, or, xor, onb, offb)



Bitwise Mask and Bit-Field Modification Modes (4 Bytes)

The MSP instruction provides dedicated bitwise masking and flag toggling operations:

- **`MSP.and` (0b011, 0xD3):** Bitwise AND mask: SYS_REG[id] $\leftarrow$ SYS_REG[id] $\&$ imm16.
- **`MSP.or` (0b100, 0xD4):** Bitwise OR mask: SYS_REG[id] $\leftarrow$ SYS_REG[id] | imm16.
- **`MSP.xor` (0b101, 0xD5):** Bitwise XOR toggle: SYS_REG[id] $\leftarrow$ SYS_REG[id] ^ imm16. Toggles specific control flags.
- **`MSP.onb` (0b110, 0xD6):** Turn bits ON: SYS_REG[id] $\leftarrow$ SYS_REG[id] | imm16. Enables specific flags (e.g. Master Interrupts MIE, Privilege PRV, Auto-Increment AIP) without disturbing other bits.
- **`MSP.offb` (0b111, 0xD7):** Turn bits OFF: SYS_REG[id] $\leftarrow$ SYS_REG[id] $\&$ $\sim$ imm16. Clears specific flags safely.
- **Instruction Length:** Occupies exactly **4 bytes** of program memory.

Sub-Operation Code Reference

Bits [2:0]	Byte 0	Prefix	Assembly Syntax	Len	Microcode Operation and Functional Purpose
000	0xD0	.ld	MSP.ld reg, imm16	4B	SYS_REG[id] ← imm16. Direct 16-bit constant loading.
001	0xD1	.ldr	MSP.ldr reg	2B	SYS_REG[id] ← R7. Fast transfer from general register R7 (alias: .ldr7).
010	0xD2	.str	MSP.str reg	2B	R7 ← SYS_REG[id]. Fast transfer into general register R7 (alias: .str7).
011	0xD3	.and	MSP.and reg, imm16	4B	SYS_REG[id] ← SYS_REG[id] & imm16. Bitwise mask AND.
100	0xD4	.or	MSP.or reg, imm16	4B	SYS_REG[id] ← SYS_REG[id] | imm16. Bitwise mask OR.
101	0xD5	.xor	MSP.xor reg, imm16	4B	SYS_REG[id] ← SYS_REG[id] ^ imm16. Bitwise toggle XOR.
110	0xD6	.onb	MSP.onb reg, imm16	4B	SYS_REG[id] ← SYS_REG[id] | imm16. Explicit bit turn-ON mask.
111	0xD7	.offb	MSP.offb reg, imm16	4B	SYS_REG[id] ← SYS_REG[id] & ∼ imm16. Explicit bit turn-OFF mask.

Map of Quantized 16-bit System Register Cells (REG_ID 0–255)

Group	Dec Cells	Hex Cells	Canonical Mnemonics	Functional Description and Mapping
IVT	0–31	0x00-0x1F	ir0_lo..ir15_hi	16 Interrupt Vectors IR0–IR15 (even: _lo, odd: _hi).
PVZ	32–59	0x20-0x3B	pv0_lo..pv13_hi	7 Privilege Protection Zones 0–6 (Start/End pairs).
MEM	60–63	0x3C-0x3F	ram_base, ram_limit	User RAM Physical Base (60, 61) and Limit (62, 63).
SYS_EXT	64–78	0x40-0x4E	sys0..sys14	15 Reserved system registers for coprocessors and timers.
Reserved	79–127	0x4F-0x7F	—	49 Reserved cells for core extensions.
FREG	128	0x80	freg (flags)	Status and Control Flags Register (ZF, CF, OF, EM, CMP, MIE, PRV).
SP	129, 130	0x81, 0x82	sp_lo, sp_hi (sp)	Hardware Stack Pointer (129: SP[15:0], 130: SP[31:16]).
PTREG	131, 132	0x83, 0x84	ptreg_lo, ptreg_hi	Streaming Base Pointer (131: PT[15:0], 132: PT[31:16]).
Reserved	133–255	0x85-0xFF	—	123 Reserved cells for hardware expansions.

Length: 2 or 4 bytes **Operands:** 1 or 2 **Mode:** Supervisor / User **Flags affected:** Modifies FREG when Cell=128; otherwise preserved

Usage Example (LIM2 Assembly)

```
; 1. Flags Register FREG (Cell 128) - without $ and without 7
MSP.ld      freg, 0x1800                ; Supervisor mode (PRV=1), AIP=1
MSP.onb     freg, 0x0200                ; Turn ON Master Interrupts (MIE = bit 9)
MSP.offb    freg, 0x0200                ; Turn OFF Master Interrupts (MIE = bit 9)

; 2. Stack pointer (sp) and Streaming Pointer register (ptreg)
MSP.ld      sp, 0x0007FFFC              ; Automatically expands into sp_lo=129 and
    sp_hi=130
MSP.ld      ptreg, 0x0001A000           ; Automatically expands into ptreg_lo=131 and
    ptreg_hi=132

; 3. Interrupt Vector I$0
MSP.ld      i$0, 0x00005000             ; Expands into i$0_lo (cell 0) and i$0_hi (
    cell 1)

; 4. Fast exchange with General-Purpose Register $7 (2 bytes each: MSP.str and MSP.ldr)
MSP.str     freg                        ; $7 = FREG (save flags state)
PUSH        $7                          ; Push saved flags onto stack
MSP.ldr     freg                        ; Restore FREG from $7
```

Operational Notes: The unified MSP architecture provides clean mnemonic syntax without requiring special prefixes. Register R7 serves as the hardware gateway for 2-byte transfers via `MSP.ldr` and `MSP.str`.

▸ 9.31 MOD — Integer modulo computation (remainder of division).

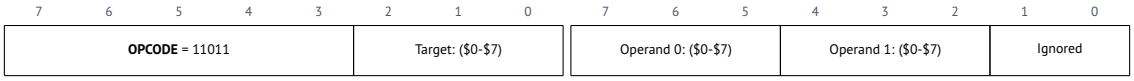
MOD

OPCODE: 11011

[Arithmetic]

Instruction Purpose:
Integer modulo computation (remainder of division).

Instruction Format Diagram



Detailed Description and Microcode Semantics

The MOD (Modulo) instruction calculates the integer remainder from dividing two 16-bit operands:

- **Data Path and ALU:** Divides the first operand `Operand 0` (A) by the second operand `Operand 1` (B) and writes the remainder into `Target` (`Target` $\leftarrow$ `Operand 0` $\pmod{\texttt{Operand 1}}$).
- **Division by Zero (Error Math):** Upon division by zero (`Operand 1` $= 0$), the ALU asserts error flag EM=1 (bit 3 of FREG) and sets zero flag ZF=1. Register `Target` receives `0x0000`, and execution traps to ISR Vector 1.
- **Zero Flag (ZF):** If the remainder is zero (exact divisibility), flag ZF=1 is asserted in hardware. This enables efficient parity and divisibility checks in pairing with JZ/JNZ.

Length: 2 bytes **Operands:** 3 **Mode:** User / Supervisor **Flags affected:** ZF (if remainder is 0), EM (on division by 0)

Usage Example (LIM2 Assembly)

```
; Compute modulo remainder: 17 mod 5 = 2
LMR  $1, 0x0011          ; $1 = 17
LMR  $2, 0x0005          ; $2 = 5
MOD  $0, $1, $2     ; $0 = 17 mod 5 = 2 ($0 = 0x0002)

; Parity test (divisor = 2):
LMR  $2, 0x0002          ; Divisor = 2
MOD  $0, $3, $2     ; $0 = $3 mod 2; if even -> ZF = 1
JZ   IsEven                ; Branch if divisible without remainder
```

Operational Notes: The MOD instruction performs integer division and saves the remainder into `Target`. If the operands divide evenly, ZF=1 is asserted. Division by zero asserts error flag EM=1.

▸ 9.32 INTR / INT — Software interrupt; invoke handler via vector IR0–IR15.

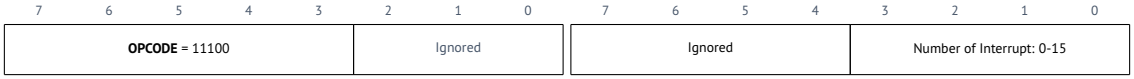
INTR / INT

OPCODE: 11100

[System]

Instruction Purpose:
Software interrupt; invoke handler via vector IR0–IR15.

Instruction Format Diagram



Detailed Description and Microcode Semantics

The INT (Interrupt) instruction programmatically invokes a system interrupt service routine:

- **Data Path and Stack:** Saves the current 32-bit program counter PC (address of the instruction following INT) onto the hardware stack: SP is decremented by 4 bytes (SP $\leftarrow$ SP $-$ 4), and the return address is written to the stack top ([SP] $\leftarrow$ PC, Little-Endian).
- **Vector Branching:** Lower 4 bits $[3:0]$ of the second instruction byte specify the interrupt vector number from 0 to 15 (IR0–IR15). The program counter is loaded from the vector register: PC $\leftarrow$ INT_VEC[Vector]. Bits $[7:4]$ of the second byte are ignored.
- **Flags and Priority:** The INTR status bit (bit 15 of FREG) is asserted to **1**, locking out nested hardware interrupts. Software interrupts execute unconditionally regardless of the master mask flag MIE.

Length: 2 bytes **Operands:** 1 (vector index) **Mode:** User / Supervisor **Flags affected:** Asserts INTR=1 in FREG

Usage Example (LIM2 Assembly)

```
; Invoke OS system service / interrupt I$1:
INT  1                    ; Push PC to stack, jump to INT_VEC1
; Software trigger of timer ISR 0:
INT  0                    ; Push PC to stack, jump to INT_VEC0
```

Operational Notes: The INT instruction is used for system call traps, OS supervisor transitions, and software event emulation. The handler returns to caller using RET.

▸ 9.33 CALL — Subroutine call: push 32-bit PC to stack and branch to address.

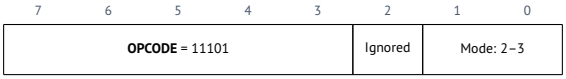
CALL

OPCODE: 11101

[Branch]

Instruction Purpose:
Subroutine call: push 32-bit PC to stack and branch to address.

Instruction Format Diagram



Detailed Description and Microcode Semantics

The CALL instruction invokes a subroutine while preserving the 32-bit return address onto the stack:

- **Data Path and Stack:** Pushes the 32-bit PC of the next instruction onto the hardware stack: SP $\leftarrow$ SP $-$ 4, [SP] $\leftarrow$ PC (Little-Endian).
- **Addressing Modes (Identical to JMP):** Lower bits $[1:0]$ of the first byte (Mode) specify the branch target:
  - **00** (0) — **Direct:** 32-bit immediate target following opcode (5 bytes total).
  - **01** (1) — **Indirect:** 32-bit address read from memory at pointer [ADDR] (5 bytes total).
  - **10** (2) — **Via PTREG:** Target address held in PTREG (1 byte).
  - **11** (3) — **Via Register Pair:** Composite address $(Source_h \ll 16) \mid Source_l$ (2 bytes).
- **Flags Affected:** The CALL instruction does not alter FREG flags.



Length: 1, 2, or 5 bytes **Operands:** 1 **Mode:** User / Supervisor **Flags affected:** Flags preserved (no effect on FREG).

Usage Example (LIM2 Assembly)

```
; 1. Direct subroutine call (Mode 0, 5 bytes):
CALL 0x00002000              ; [SP-4] = PC, SP -= 4, PC = 0x00002000
; 2. Fast call via pointer register PTREG (Mode 2, 1 byte):
CALL [$PTREG]              ; [SP-4] = PC, SP -= 4, PC = PTREG
; 3. Call via address from register pair (Mode 3, 2 bytes):
CALL [$0:$1]           ; [SP-4] = PC, SP -= 4, PC = ($0 << 16) | $1
```

Operational Notes: The CALL instruction pushes the 32-bit return address onto the stack SP and branches to the subroutine. The subroutine returns using RET.

▸ 9.34 RET — Return from subroutine with 32-bit PC restored from stack.

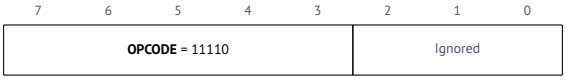
RET

OPCODE: 11110

[Branch]

Instruction Purpose:
Return from subroutine with 32-bit PC restored from stack.

Instruction Format Diagram



Detailed Description and Microcode Semantics

The RET (Return) instruction returns control from a previously called subroutine or interrupt service routine:

- **Data Path and Stack:** Reads the 32-bit return address from memory at the top of the stack: PC $\leftarrow$ [SP] (Little-Endian). The stack pointer is then incremented by 4 bytes: SP $\leftarrow$ SP $+ 4$.
- **Operating Principle:** The execution core branches to the restored PC address, resuming immediately after the originating CALL or INT instruction. If exiting an interrupt, INTR is atomically cleared to 0.
- **Flags Affected:** The RET instruction does not modify FREG arithmetic flags. Any flags established by the called routine are retained.

Length: 1 byte **Operands:** None **Mode:** User / Supervisor **Flags affected:** Flags preserved (no effect on FREG).

Usage Example (LIM2 Assembly)

```
CalculateSum:
    ADD  $0, $0, $1     ; Function body: $0 = $0 + $1
    RET                 ; Restore PC from stack: PC = [SP], SP += 4
```

Operational Notes: The RET instruction is the pairing counterpart to CALL and INT. It restores the 32-bit program counter PC from the top of the hardware stack.

▸ 9.35 HALT — Hardware clock halt until interrupt or peripheral data ready.

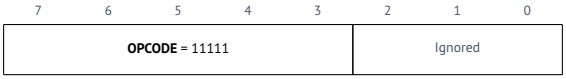
HALT

OPCODE: 11111

[Control]

Instruction Purpose:
Hardware clock halt until interrupt or peripheral data ready.

Instruction Format Diagram



Detailed Description and Microcode Semantics

The HALT instruction transitions the core into a low-power quiescent wait state:

- **Operating Principle:** Disables the core clock distribution network. The microcode sequencer and ALU cease instruction execution, and bus instruction fetch cycles are halted.
- **Wake-Up Conditions (Wakeup):** The processor resumes clocking and execution upon either of two hardware events:
  1. **Interrupt Event:** Assertion of an external hardware interrupt (provided MIE=1) or non-maskable interrupt (RDI).
  2. **Data Ready Strobe:** Assertion of the external peripheral/memory bus ready line (RF / Data Ready).
- **Clock Resumption (CLK):** Upon wake-up, the clock generator resumes pulsing at the mode and frequency designated by the CLK field (bits 7:6 of FREG).
- **State Integrity:** General-purpose registers ($0–$7), pointers PC, SP, PT, and status register FREG remain fully preserved.

Length: 1 byte **Operands:** None **Mode:** User / Supervisor **Flags affected:** Flags preserved (no effect on FREG).

Usage Example (LIM2 Assembly)

```
; Wait for external interrupt or bus data readiness:
HALT                        ; Shut down core clocking (low-power wait)
; Wakes up upon interrupt / Data Ready at clock rate defined by CLK:
LMR  $0, [$PTREG]      ; Read received data immediately upon wake-up
```

Operational Notes: The HALT instruction provides deterministic low-power standby awaiting I/O events or interrupts. Execution resumes using the clock source configured in CLK of FREG.

10 Interrupt System and Exception Handling

▸ 10.1 Microarchitectural Interrupt Mechanism

The LIM2 processor implements a strictly deterministic mechanism for servicing software and hardware interrupts, as well as internal core exception traps (Faults). The architecture defines **16 system vectors** (`INT_VEC0`–`INT_VEC15`), each holding an independent 32-bit physical entry address for its corresponding Interrupt Service Routine (ISR).

Hardware Interrupt Trap Sequence (Trap Cycle)

Upon occurrence of an external hardware event, trigger of an internal core exception, or execution of the `INT n` instruction, the processor executes the following hardware actions within 2 machine cycles:

1. **Pushing Return Address to Stack:** The current value of the program counter PC (pointing to the subsequent instruction) is pushed onto the top of the hardware stack:

$$\text{SP} \leftarrow \text{SP} - 4, \quad [\text{SP}] \leftarrow \text{PC} \quad \text{(Little-Endian)}$$

2. **Nested Interrupt Lockout:** The hardware status bit `INTR` (bit 15 of register FREG) is automatically asserted to **1**. This prevents race conditions and stack frame corruption from nested interrupt requests.
3. **Target Vector Vectoring:** The program counter PC is forcibly loaded with the target entry address from the system vector register: $\text{PC} \leftarrow \texttt{INT_VEC}[n]$.
4. **Return from Interrupt:** Termination of the ISR is performed by executing the `RET` instruction. It pops the return address from the top of the stack ($\text{PC} \leftarrow [\text{SP}]$, $\text{SP} \leftarrow \text{SP} + 4$) and automatically clears the `INTR` flag back to **0**, re-enabling interrupt servicing.

▸ 10.2 Vector Table and Core Exceptions

At power-on, all 16 vectors are initialized to zero (`0x00000000`). The operating system or primary bootloader configures them during early startup using `MSP.ld` instructions. The LIM2 architecture strictly partitions the first 8 vectors for internal core hardware exceptions, while allocating vectors 8–15 entirely for operating system services:

No.	Vector	Mnemonic	Class	Source and Functional Description
0	0x00	**RESET**	System Boot	Hardware reset and core cold-boot entry point.
1	0x01	**DIV_ZERO**	Math Fault	Division by zero during DIV or MOD execution (EM=1).
2	0x02	**ILLEGAL_OP**	Core Fault	Attempt to execute undefined opcode or illegal operand mode.
3	0x03	**PROT_FAULT**	Security Fault	Violation of PZV zones or User-mode access beyond MEM_VEC.
4	0x04	**STACK_FAULT**	Memory Fault	Stack pointer SP exceeding allocated stack boundary.
5	0x05	**BUS_TIMEOUT**	Bus Fault	Bus ready timeout (RF) with DWR=0 exceeding cycle limit.
6	0x06	**SYS_TIMER**	Hardware Core	Periodic hardware system timer tick interrupt.
7	0x07	**DATA_READY**	Hardware Core	External bus data ready signal (RF / wake-up from HALT).
8	0x08	**SW_INT8**	OS Service	Software interrupt: OS core services / I/O dispatch.
9	0x09	**SW_INT9**	OS Service	Software interrupt: Process scheduler and thread context switch.
10	0x0A	**SW_INT10**	OS Service	Software interrupt: File subsystem and block storage I/O.
11	0x0B	**SW_INT11**	OS Service	Software interrupt: Inter-process communication (IPC) messaging.
12	0x0C	**TIMER1**	Hardware Timer	Timer 1 match interrupt (frequency generator / real-time clock tick).
13	0x0D	**TIMER2**	Hardware Timer	Timer 2 periodic reload timeout interrupt.
14	0x0E	**TIMER3**	Hardware Timer	Timer 3 one-shot delay expiration interrupt.
15	0x0F	**TIMER4**	Hardware Timer	Timer 4 instruction quantum preemptive scheduler interrupt.

▸ 10.3 Partitioning of Core Faults, Software Interrupts, and Hardware Timers

- **Vectors 0–7 (Internal Core Events and Faults):** Reserved by hardware for critical processor events, arithmetic faults, memory protection, and bus timeout. They are triggered unconditionally by hardware, irrespective of the master mask flag `MIE`.
- **Vectors 8–11 (Operating System Software Services):** Freely defined by the OS architect to implement system call APIs, device drivers, and user services. Invoked programmatically via the `INT n` instruction.
- **Vectors 12–15 (Dedicated Core Hardware Timers):** Static vector assignments for the four hardware timers: IRQ 12 (Timer 1 match / real-time tick), IRQ 13 (Timer 2 periodic reload), IRQ 14 (Timer 3 one-shot expiration), and IRQ 15 (Timer 4 instruction quantum scheduler preemption).

11 Memory Organization and Philosophy of Freedom

11.1 Fundamental Principle: Absolute Architectural Freedom

The primary objective and guiding architectural philosophy of the LIM2 processor is **absolute freedom for the system architect**. The processor deliberately imposes no silicon dogmas on board designers regarding board layout, RAM location, memory capacity, or peripheral address ranges.

In LIM2 silicon, there are no hardwired address maps or mandatory address translation tables. To the execution engine, the entire 4 GB (2^{32} bytes) physical address space represents a unified, unconstrained, and homogeneous flat canvas.

The Sole Hardware Boot Axiom: Entry Point 0x00000000

The only assumption made by the processor core upon de-assertion of the RESET line:

Executable primary bootloader instructions reside at physical address `0x00000000`.

- **Arbitrary Boot Media:** What physically resides at `0x00000000` is strictly up to the board designer. It may be an inexpensive 512-byte EEPROM with a tiny hardware loader, serial Flash, battery-backed static RAM, or system memory preloaded by an in-circuit debugger.
- **Direct Execution from Primary Media:** Upon reset, the core immediately begins fetching instructions directly from `0x00000000`. If the boot media is slow, the processor transparently executes bus wait cycles (via RF pin with DWR=0). This represents a deliberate engineering trade-off: maximal silicon simplicity in exchange for unbounded architectural flexibility.
- **Self-Configuring Platform:** The bootloader code organizes the machine environment: it knows the physical location of RAM (whether at `0x00001000`, `0x20000000`, or elsewhere), initializes the stack pointer SP, populates interrupt vectors, relocates itself into fast RAM if necessary, or directly boots the operating system.

11.2 Direct Memory-Mapped I/O (MMIO) and Autonomous Controllers

The LIM2 architecture contains no isolated I/O instructions (such as `IN`/`OUT` in x86). All peripheral interfacing is structured around **Memory-Mapped I/O (MMIO)**:

- **Direct Bus Integration:** Every peripheral connects to the system bus and decodes its own dedicated physical address (or range) as determined by the board designer.
- **Autonomous Device Controllers:** For robustness and decoupled operation, peripheral interfaces (UART, display controllers, storage devices, networking chips, timers) typically incorporate dedicated controller hardware with local command registers, status registers, and FIFO buffers.
- **Standard Transfer Instructions:** The processor communicates with device controllers using standard memory read and write instructions:

```
LRM [DEV_ADDR], $0   ; Direct write of command or data byte to device controller
 LMR $0, [DEV_ADDR + 2]   ; Read device status register or received data buffer
```

11.3 Security Model: Dynamic Protection Without Hardcoded Constants

Because memory topology is completely flexible, the core incorporates no hardwired silicon protection boundaries. Security domains are configured dynamically via software using `MSP.ld` instructions adapted to the physical platform:

- **7 Supervisor Windows (PZV_VEC):** Address intervals $[\text{PZV_VEC}_{2k} \ldots \text{PZV_VEC}_{2k+1}]$. Elevation to supervisor mode (PRV $= 1$) is permitted only if the current program counter PC falls within one of these windows. An unauthorized attempt to elevate privileges outside these zones triggers a PROT_FAULT exception (Vector 3).
- **User RAM Boundaries (MEM_VEC):** In user mode (PRV $= 0$), all memory references are strictly confined to the range [MEM_VEC_START ... MEM_VEC_END]. Any out-of-bounds access immediately faults with PROT_FAULT, safeguarding device registers and kernel code.

12 Practical System Assembly Examples

▸ 12.1 Architectural Conventions and Programming Model

Software engineering on the LIM2 platform adheres to a well-defined register model and calling convention. The official LIM2 assembler supports symbolic labels (`Label:`), immediate numeric constants in hexadecimal (`0x...`) and decimal notation, and symbolic system register aliases.

Basic Rules and Conventions of LIM2 Assembly

- **Label Syntax:** Labels are declared with a trailing colon (`Entry:`, `LoopHead:`). In branching instructions (JMP, JZ, CALL), the label name is resolved by the assembler into a 32-bit absolute address.
- **Register Allocation Convention (ABI):**
  - **$0 ... $3** — Scratch registers for intermediate arithmetic and function arguments / return values.
  - **$4 ... $7** — Callee-saved general-purpose data registers (must be preserved across calls via PUSH / POP).
  - **$PT** (**PTREG**) — Base memory addressing pointer for streaming block transfers with auto-increment (AIP).
  - **$SP** (Stack Pointer) — Hardware stack pointer (grows downwards; 4 bytes for PC addresses, 2 bytes for GPRs).
- **Structure of Interrupt Handlers (ISR):** An interrupt handler must preserve all modified general-purpose registers onto the stack via PUSH and restore them prior to exiting via POP. ISR termination is performed with the RET instruction, which pops PC and atomically resets INTR $\rightarrow$ 0.

▸ 12.2 Example 1: Complete System Cold-Boot Initialization (Startup / Bootloader)

Upon reset, the processor begins execution at address `0x00000000` in supervisor mode (PRV=1). The early initialization code (BIOS) configures the core environment, populates interrupt vectors, establishes User RAM boundaries, and transitions execution to the user application.

Listing: Cold Boot and Hardware Initialization (startup.asm)

```
; ==============================================================================
; SYSTEM RESET ENTRY POINT (RESET ENTRY)
; ==============================================================================
Entry:
    MSP.ld      $SP, 0x0007FFFC          ; Initialize stack pointer (top of RAM 0x0007FFFC
    )

    ; Configure Interrupt Vector Table (INT_VEC0 - INT_VEC15):
    MSP.ld      $INT_VEC0, Entry         ; Vector 0: Master Reset / Reboot
    MSP.ld      $INT_VEC1, IsrDivZero    ; Vector 1: Divide by Zero Exception
    MSP.ld      $INT_VEC2, IsrIllegalOp  ; Vector 2: Undefined / Illegal Instruction
    MSP.ld      $INT_VEC3, IsrProtFault  ; Vector 3: Memory Protection / PZV Fault
    MSP.ld      $INT_VEC6, IsrTimer      ; Vector 6: Periodic System Timer Tick
    MSP.ld      $INT_VEC7, IsrDataReady  ; Vector 7: Peripheral Data Ready
    MSP.ld      $INT_VEC15, SyscallEntry ; Vector 15: Kernel System Call Dispatcher

    ; Configure Supervisor Zone PZV_0 (Kernel Space: 0x00000000 - 0x0000FFFF):
    MSP.ld      $PZV_VEC0_S, 0x00000000  ; Supervisor Zone 0 Start Address
    MSP.ld      $PZV_VEC0_E, 0x0000FFFF  ; Supervisor Zone 0 End Address

    ; Configure User RAM Space (User RAM: 0x00010000 - 0x00077FFF):
    MSP.ld      $MEM_VEC_S, 0x00010000   ; User RAM Start Boundary (64 KB)
    MSP.ld      $MEM_VEC_E, 0x00077FFF   ; User RAM End Boundary (~480 KB)

    ; Configure FREG: Auto-inc AIP=1, Interrupts MIE=1, Clock CLK=00:
    MSP.ld      $FREG, 0x1200            ; Load base template: AIP=1, MIE=1
    MSP.and     $FREG, 0xF7FF            ; Clear PRV bit (bit 11) -> Drop to User Mode
    JMP         UserEntry                   ; Jump to user application payload in RAM

; ==============================================================================
; USER SPACE PAYLOAD (USER SPACE APPLICATION)
; ==============================================================================
UserEntry:
    LMR         $0, 0x00000005           ; $0 = 5 (dividend)
    LMR         $1, 0x00000000           ; $1 = 0 (divisor)
    DIV         $0, $1                ; Division by zero attempt -> Hardware trap to
    IsrDivZero!

AppLoop:
    HALT                                 ; Power-save wait for timer interrupt
    JMP         AppLoop                  ; Repeat loop after wake-up
```

▸ 12.3 Example 2: Hardware Divide-by-Zero Exception Handler

When executing DIV or MOD with a zero divisor, the ALU asserts the math error flag EM (bit 3 of FREG) and vectors to INT_VEC1. The handler resolves the fault state and returns control.

Listing: Divide-by-Zero Exception Handler (div_zero.asm)

```
; ============================================================================
; DIVIDE-BY-ZERO EXCEPTION HANDLER (ISR FOR VECTOR 1)
; ============================================================================
IsrDivZero:
    PUSH        $0                  ; Preserve $0 on stack
    PUSH        $1                  ; Preserve $1 on stack
    LMR         $0, 0x0000FFFF      ; Set safe default fallback value (0xFFFF)
    MSP.and     $FREG, 0xFFF7       ; Clear EM flag (bit 3 of FREG) using AND mask
    POP         $1                  ; Restore registers
    POP         $0                  ;
    RET                               ; Return to caller (clears INTR -> 0)
```

▸ 12.4 Example 3: System Interval Timer Handler (System Timer Tick)

The interval timer generates periodic interrupts via Vector 6 (INT_VEC6). The handler increments a 32-bit uptime counter located in system kernel memory.

Listing: Interval Timer Interrupt Handler (timer.asm)

```
; ============================================================================
; SYSTEM TIMER INTERRUPT HANDLER (ISR FOR VECTOR 6)
; ============================================================================
IsrTimer:
    PUSH        $0                  ; Preserve registers
    PUSH        $1                  ;
    LMR         $0, [0x00004100]    ; Load low word Time_Low
    INC         $0                  ; Increment tick count
    LRM         [0x00004100], $0    ; Store updated Time_Low
    JNZ         TimerDone             ; If no overflow -> exit
    LMR         $1, [0x00004102]    ; Load high word Time_High
    INC         $1                  ; Cascaded increment
    LRM         [0x00004102], $1    ; Store Time_High
TimerDone:
    POP         $1                  ; Restore context
    POP         $0                  ;
    RET                               ; Return to interrupted task
```

▸ 12.5 Example 4: Software Operating System Trap (Syscall)

User applications request kernel services (such as UART character transmission or scheduler yield) via the `INT 15` instruction. The function index is passed in `$0`, and arguments are passed in `$1`.

Listing: Operating System Syscall Dispatcher (syscall.asm)

```
; ==============================================================================
; USER CALL EXAMPLE: TRANSMIT CHARACTER 'A' OVER UART
; ==============================================================================
UserPrintChar:
    LMR         $0, 0x00000001          ; $0 = 1 (Function code: SYS_WRITE_UART)
    LMR         $1, 0x00000041          ; $1 = ASCII 'A' (0x41)
    INT         15                        ; Invoke kernel supervisor via Vector 15
    ; Continue user execution after kernel returns...

; ==============================================================================
; KERNEL SYSTEM CALL DISPATCHER (VECTOR 15)
; ==============================================================================
SyscallEntry:
    PUSH        $2                      ; Preserve scratch register
    LMR         $2, 0x00000001          ; Function 1 match constant (SYS_WRITE_UART)
    CMP         $0, $2                ; Compare requested code with 1
    JZ          ServiceUartWrite            ; If $0 == 1 -> dispatch to UART write

    LMR         $2, 0x00000002          ; Function 2 match constant (SYS_YIELD)
    CMP         $0, $2                ; Compare
    JZ          ServiceYield                ; If $0 == 2 -> yield execution

    LMR         $0, 0x0000FFFF          ; Otherwise: return error code 0xFFFF
    JMP         SyscallExit                 ;

ServiceUartWrite:
    LRM         [0xFFFF0000], $1        ; Physical store to MMIO UART transmitter
    LMR         $0, 0x00000000          ; Success code: 0 (SUCCESS)
    JMP         SyscallExit                 ;

ServiceYield:
    HALT                                  ; Power-save wait until next timer interrupt
    LMR         $0, 0x00000000          ; Success code

SyscallExit:
    POP         $2                      ; Restore register
    RET                                   ; Return to User Mode
```

▸ 12.6 Example 5: High-Speed Memory Block Copy (Memcpy)

Leveraging the hardware auto-increment of the pointer register PT (AIP = 1), the LIM2 processor executes high-throughput streaming memory transfers.

Listing: Fast Memory Block Copy (memcpy.asm)

```
; ============================================================================
; MEMORY BLOCK COPY SUBROUTINE: Memcpy(Dest, Src, Count)
; Parameters:
;   $0 - Destination address (Dest),      $1 - Source address (Src)
;   $2 - Number of 16-bit words to copy (Count)
; ============================================================================
Memcpy:
    LMR         $3, 0x00000000          ; $3 = 0
    CMP         $2, $3                  ; Check if Count == 0
    JZ          MemcpyDone                  ; If Count == 0 -> exit early

CopyLoop:
    MSP.ld      $PTREG, $1              ; Set PT to source address
    LMR         $4, [$PTREG]            ; $4 = *Src (auto-increment PT += 2)
    MSP.ld      $1, $PTREG              ; Save updated Src address

    MSP.ld      $PTREG, $0              ; Set PT to destination address
    LRM         [$PTREG], $4            ; *Dest = $4 (auto-increment PT += 2)
    MSP.ld      $0, $PTREG              ; Save updated Dest address

    DEC         $2                      ; Count = Count - 1 (updates ZF flag)
    JNZ         CopyLoop                    ; If Count != 0 -> continue loop

MemcpyDone:
    RET                                 ; Return from subroutine
```

13 Architectural Roadmap and Reserved Modules (TODO)

The LIM2 architecture is engineered for future scalability without compromising backward binary compatibility. The following hardware modules and architectural extensions are planned or reserved for future silicon revisions:

LIM2 Architectural Extension Roadmap (Revision 2.x / 3.0)

- **[TODO] Direct Memory Access Controller (DMA Engine):** Development of an autonomous background data transfer engine connecting system RAM with high-speed peripherals (SPI, network controller, frame buffer) without stalling the core execution pipeline.
- **[TODO] Floating-Point Coprocessor Interface (FPU Coprocessor Bus):** Dedicated hardware

expansion bus for single- and double-precision IEEE 754 floating-point coprocessors, invoked via reserved system coprocessor vectors.

- **[TODO] On-Chip Debugger (JTAG / SWD Interface):** Hardware TAP controller providing non-intrusive hardware breakpoints, single-step execution tracing, and live register file inspection without stopping the master clock generator.
- **[TODO] Streaming String and Vector Extensions (Stream Extensions):** Hardware circular buffer acceleration and vectorized byte sequence scanning for network packet parsing and high-speed text processing.